SATA 1D: Algorithm, Implementation and Verification

Position-dependent correction of the topographic bulk residual ΔC_0
in multichannel azimuth reconstruction

Lucas Ayub Rodrigues Tuler
DLR–DAAD Fellowship No. 734 — Microwaves and Radar Institute (DLR–HR)

September 2026

Abstract

This note documents the one-dimensional SATA correction of the bulk topographic residual ΔC_0 as it is now implemented in the `sata1d` package, and the verification that was run on it. The correction has two stages. Stage A builds, for every channel, a map $\Delta C_0[x]$ that holds the residual of each scatterer only on the azimuth cells where that scatterer's energy appears in the sub-aperture spectra, and zero elsewhere. Stage B is the sub-aperture kernel: every Doppler bin of every sub-aperture is mapped through the beam angle onto an azimuth cell, reads the map there, and is rotated by the corresponding phase. Each equation is derived from the geometry in the order the code executes it, and each step is shown next to the code that implements it. The verification consists of eleven tests, from the algebra of the kernel to multi-target reconstructions compared against an oracle that corrects every target with its own residual. All ten pass/fail tests pass. For a single target the azimuth ambiguity returns from between -23 and -9 dB to -39.4 dB, the level of the reference, at every height up to 400 m; on multi-target scenes the correction matches the oracle to within 0.3 dB whenever the scatterers are further apart than the SATA resolution $\delta x \simeq 309$ m. The verification also found an inconsistency in the per-sub-band kernel: its frequency axis was f_k+fftfreq with positions measured from β_k. That axis has been replaced by the absolute, f_k-centred axis on the broadside grid ($\beta_{\text{img}} = 0$) prescribed in Section 4.2 of the report.

Contents

Notation **3**

1 The Problem SATA Solves **4**

2 Stage B: the Sub-Aperture Kernel **5**
2.1 Sub-aperture length, hop and FFT size . . . 5
2.2 Window and constant overlap-add . . . 5
2.3 The ruler: from a Doppler bin to an azimuth cell . . . 6
2.4 Folded Doppler: why the target also appears $\pm X$ cells away . . . 7
2.5 The rotation . . . 7

3 Stage A: the ΔC_0 Map **8**
3.1 A1 — the residual of each scatterer . . . 9
3.2 A2 — where the scatterer sits on the channel line . . . 9
3.3 A3 — the footprint: how far a scatterer's energy spreads . . . 9
3.4 A4 — alias layers . . . 10
3.5 A5 — drawing the map . . . 10

4 Whole Band and per Sub-Band **12**
4.1 What changes 12
4.2 The sub-band axis 13
4.3 Correction of the previous per-sub-band axis 14

5 Verification **16**
5.1 Full Geometry Sweep: Scenes with Several Targets 19
5.2 Consistency with the report 27

6 Conclusions **27**

A Running the Code **28**

Notation

Symbol	Meaning	Value in the examples
λ, v	wavelength, sensor velocity	0.25 m, 7688.5 m/s
r, r_0	reference slant range (ruler and δx)	766.2 km
θ	look angle at the reference point	20.0°
PRF, $\mathrm{PRF}_{\mathrm{ch}}$	system PRF, PRF of one channel (= PRF/N_{rx})	2000 Hz, 500 Hz
N_{rx}	number of receive channels	4
$b_{at,i}$, $b_{xt,i}$	along-/cross-track baseline of channel i	7.69 m steps; $[63.7, 27.0, 4.1, 1.7]$ m
$d_s = v/\mathrm{PRF}_{\mathrm{ch}}$	azimuth cell of one channel	15.38 m
$N_{a,ch}$	samples per channel line	3072
δx	SATA (sub-aperture) resolution	309.5 m
$T_{\mathrm{sub,eff}}$, hop, N_{zp}	sub-aperture length, hop, FFT length	20, 10, 128 samples
f, ϕ	bin frequency (absolute label), baseband bin frequency	
$\beta(f)$	beam (squint) angle belonging to f	
$x(f)$	cell offset of bin f from the window centre (the *ruler*)	0.81 cells/Hz
c_w	centre cell of window w	
x_t	cell of a scatterer on the channel line	1537 (channel 0)
h_w	half-width of a scatterer's footprint in the map	82 cells
X	cell shift of one $\mathrm{PRF}_{\mathrm{ch}}$ fold	405.06 cells
B	Doppler bandwidth of a target	1281 Hz
f_k, β_k	centre frequency and squint of output sub-band k	± 250, ± 750 Hz
$C_{0,i}(p)$	bulk coefficient of channel i for point p	
$\Delta C_{0,i}(p)$	residual $C_{0,i}(p) - C_{0,i}(p_{\mathrm{ref}})$	-5.47 cm (ch. 0, $h = 240$ m)

1 The Problem SATA Solves

The residual. The reconstruction filters of channel i are built from the coefficients of the reference point p_{ref}, the point on the reference surface at the same slant range and azimuth. The channel transfer function carries the bulk term $e^{-j2\pi C_{0,i}/\lambda}$ (`filter_matrix`, `reconstruction.py`), so a scatterer p at height h leaves in the data the factor

$$e^{-j2\pi C_{0,i}(p)/\lambda} = e^{-j2\pi C_{0,i}(p_{\mathrm{ref}})/\lambda}\, e^{-j2\pi \Delta C_{0,i}(p)/\lambda}, \qquad \Delta C_{0,i}(p) = C_{0,i}(p) - C_{0,i}(p_{\mathrm{ref}}), \tag{1}$$

of which the filter removes only the first. For small heights the residual has the closed form of the interferometric height sensitivity (report, Eq. 37),

$$\Delta C_{0,i} \simeq -\frac{b_{xt,i}\,\Delta h}{r_0 \tan\theta}. \tag{2}$$

For channel 0 ($b_{xt} = 63.7\,\mathrm{m}$) and $h = 240\,\mathrm{m}$, Eq. (2) gives $-5.47\,\mathrm{cm}$, a phase of $2\pi\Delta C_0/\lambda = -78.8°$; for channel 3 ($b_{xt} = 1.7\,\mathrm{m}$) it is $-0.14\,\mathrm{cm}$ ($-2.0°$). What matters for the reconstruction is that the phases *differ* between channels: the unmixing of the aliased sub-bands is then incomplete and the missing fraction reappears as azimuth ambiguities (report, Eqs. 39–42).

Why one multiplication is not enough. If the scene had a single height, multiplying each channel by $e^{+j2\pi\Delta C_{0,i}/\lambda}$ would remove the residual exactly. But $\Delta C_{0,i}$ depends on the scatterer, and while the beam sweeps along azimuth the same channel sample receives echoes from many scatterers. The phase to remove therefore has to be selected *per scatterer*. SATA does that without focusing the line: in a short sub-aperture, Doppler frequency is equivalent to beam angle and therefore to azimuth position, so each Doppler bin can be given the residual of the scatterer that sits at the position the bin looks at.

Overview. Figure 1 places SATA in the processing chain. It acts on each channel line separately, before the reconstruction, and it has two stages that always run in the same order: Stage A builds the map $\Delta C_0[x]$ once per channel (Section 3), then Stage B runs the sub-aperture loop that reads it (Section 2). The kernel is explained first because the map is defined by what the kernel reads.

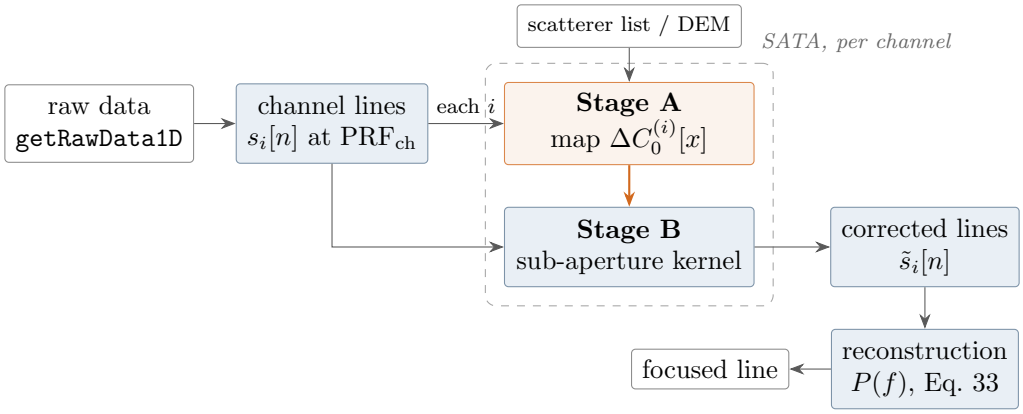


Figure 1: Place of SATA in the chain. Stage A needs the scatterers (in simulation, the scene list; with real data, a DEM). Stage B reads the map and rotates the Doppler bins. Whole band: one corrected set feeds all sub-bands of the reconstruction. Per sub-band: the two stages run once for every output sub-band k (Section 4).

2 Stage B: the Sub-Aperture Kernel

Figure 2 shows the loop with the line of code that executes each box. The subsections follow the boxes from left to right.

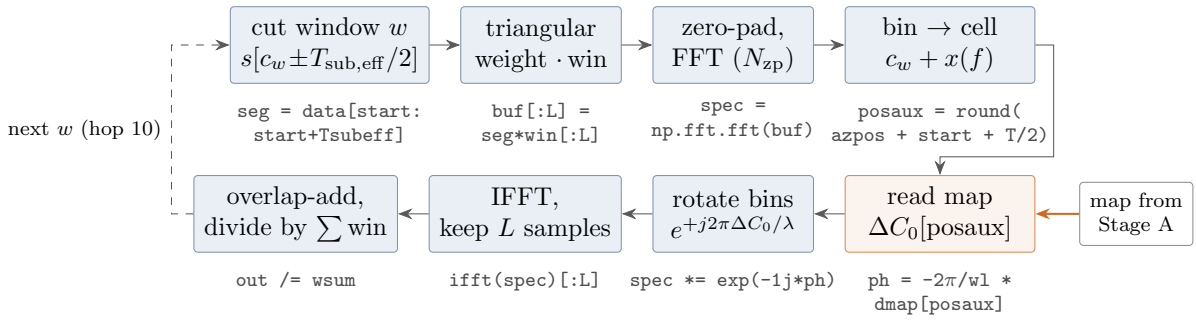


Figure 2: The kernel `sata_1d` for one channel line. Grey text: the statement that executes each box. The orange path is the only place where the map enters.

2.1 Sub-aperture length, hop and FFT size

The sub-aperture must be short enough for the residual to be constant over the ground patch it sees, and long enough for its Doppler resolution to separate positions. The classical topography-accommodation length is (report, Eq. 44)

$$\delta x = \sqrt{\frac{\lambda\, r_{\mathrm{ref}}}{2}}, \qquad T_{\mathrm{sub,eff}} = 2\left\lfloor \frac{\delta x\, \mathrm{PRF}_{\mathrm{ch}}}{2v} \right\rfloor, \qquad \mathrm{hop} = \frac{T_{\mathrm{sub,eff}}}{2}, \qquad N_{\mathrm{zp}} = 2^{\lceil \log_2 T_{\mathrm{sub,eff}} \rceil} \cdot \mathrm{osf}. \tag{3}$$

The rate is that of the line actually processed, $\mathrm{PRF}_{\mathrm{ch}}$ (report, Section 4.2, point 1). Numerically, $\delta x = \sqrt{0.25 \cdot 766\,200/2} = 309.5\,\mathrm{m}$, which is $\delta x\, \mathrm{PRF}_{\mathrm{ch}}/v = 20.1$ samples, so $T_{\mathrm{sub,eff}} = 20$, hop $= 10$ and, with osf $= 4$, $N_{\mathrm{zp}} = 32 \cdot 4 = 128$. The bin spacing is $\mathrm{PRF}_{\mathrm{ch}}/N_{\mathrm{zp}} = 3.906\,\mathrm{Hz}$.

`sata1d/sata.py, lines 25--30`

```
25 def subaperture_sizing(rref, prf, v, wl, sata_osf=1):
26     """Sub-aperture length Tsubeff (even), hop Tsub (50 % overlap), FFT size Nzp."""
27     deltax = np.sqrt(wl * rref / 2.0)                    # SATA resolution on ground [m]
28     Tsubeff = int(np.round(deltax * prf / v * 0.5) * 2)
29     Nzp = int(2 ** np.ceil(np.log2(Tsubeff)) * sata_osf)
30     return Tsubeff, Tsubeff // 2, Nzp
```

2.2 Window and constant overlap-add

The analysis window is triangular, $\mathrm{win}[n] = n/(T_{\mathrm{sub,eff}}/2-1)$ on the rising half and mirrored on the falling half. At 50 % overlap the shifted windows sum to a constant, $\sum_w \mathrm{win}[n - w\cdot\mathrm{hop}] = \mathrm{const}$ (constant overlap-add, COLA), and the loop divides the accumulated output by that sum:

$$\tilde{s}[n] = \frac{\sum_w \mathrm{IFFT}\{S_w \cdot R_w\}[n - w\,\mathrm{hop}]}{\sum_w \mathrm{win}[n - w\,\mathrm{hop}]}, \qquad S_w = \mathrm{FFT}_{N_{\mathrm{zp}}}\{s \cdot \mathrm{win}\}, \tag{4}$$

where R_w is the per-bin rotation of Section 2.5. With $R_w \equiv 1$ the IFFT returns the windowed segment $s \cdot \mathrm{win}$ and Eq. (4) reduces to $\tilde{s} = s$: a null map reproduces the input to machine precision (test T1).

`sata1d/sata.py, lines 46--48`

```
46 def triangular_window(Tsubeff):
```

```
    half = np.arange(Tsubeff // 2) / (Tsubeff // 2 - 1)
    return np.concatenate((half, half[::-1]))
```

2.3 The ruler: from a Doppler bin to an azimuth cell

This is the step that makes the correction position-dependent. Take a scatterer at cell x_t and a window centred at cell c_w (Fig. 3). The along-track distance between the phase centre and the scatterer is $d = (x_t - c_w)\, d_s$ and the range is $R = \sqrt{r^2 + d^2}$. The two-way Doppler is

$$f = -\frac{2}{\lambda}\frac{dR}{dt} = \frac{2v}{\lambda}\frac{d}{R} = \frac{2v}{\lambda}\sin\beta, \qquad \tan\beta = \frac{d}{r}. \tag{5}$$

The kernel inverts Eq. (5). A bin with frequency f belongs to the angle $\beta(f)$ and hence to the distance $d = r\tan\beta(f)$, that is, to the cell

$$\beta(f) = \arcsin\left(\frac{\lambda f}{2v}\right), \qquad x(f) = r\left[\tan\beta(f) - \tan\beta_{\text{img}}\right]\frac{\text{PRF}_{\text{ch}}}{v}, \qquad \text{posaux}(f) = c_w + x(f), \tag{6}$$

which is Eq. 45 of the report, with $\beta_{\text{img}} = 0$ for the broadside grid used here. The decisive property is that the ruler is the exact inverse of the physics that placed the energy. Wherever the window is, the bin that holds the scatterer's energy satisfies $c_w + x(f) = x_t$: *the peak always points at the scatterer's own cell*, even though it moves through the spectrum from window to window.

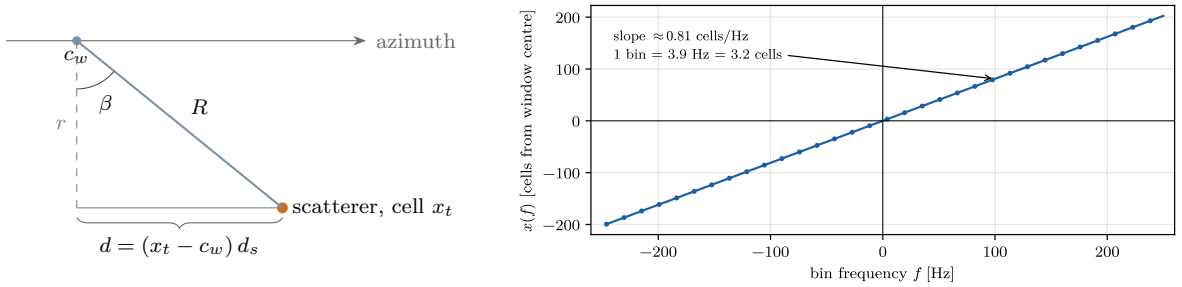


Figure 3: Left: geometry of Eq. (5); the scatterer seen from the window centre under the squint β. Right: the ruler $x(f)$ of Eq. (6) for the 128 bins of the kernel. It is almost linear, $x(f) \simeq (r\lambda\text{PRF}_{\text{ch}}/2v^2)\, f = 0.81$ cells/Hz, so the band $\pm\text{PRF}_{\text{ch}}/2$ spans ± 203 cells around the window centre and one bin spans 3.2 cells.

In code, `subaperture_axis` builds the bin frequencies in FFT order (with the Nyquist bin labelled $+\text{PRF}_{\text{ch}}/2$) and their cell offsets:

sata1d/sata.py, lines 33--43

```
def subaperture_axis(prf, Nzp, v, wl, r, squint=0.0):
    """Doppler of each FFT bin (np.fft.fft order) and the cell offset it points to:
    f -> beta = asin(wl f / 2v) -> r (tan beta - tan squint) / v * prf."""
    fc = 2.0 * v / wl * np.sin(squint)
    pfc = np.round(np.mod(fc, prf) * Nzp / prf)
    dfc = np.round(fc * Nzp / prf) * prf / Nzp
    fsub = np.arange(Nzp) * prf / Nzp - prf * 0.5 + dfc
    fsub[0] = prf * 0.5 + dfc
    fsub = np.roll(fsub, int(Nzp / 2 + pfc))
    betasub = np.arcsin(np.clip(wl * fsub / (2.0 * v), -1.0, 1.0))
    return fsub, r * (np.tan(betasub) - np.tan(squint)) / v * prf
```

2.4 Folded Doppler: why the target also appears $\pm X$ cells away

A channel is sampled at $\mathrm{PRF}_{\mathrm{ch}} = 500\,\mathrm{Hz}$, but a target sweeps a Doppler band $B = 4v\sin(\theta_{az}/2)/\lambda = 1281\,\mathrm{Hz}$ (θ_{az}: azimuth beamwidth). A true Doppler f outside $[-\mathrm{PRF}_{\mathrm{ch}}/2, \mathrm{PRF}_{\mathrm{ch}}/2)$ is therefore stored in the bin labelled $f + m\,\mathrm{PRF}_{\mathrm{ch}}$ for the integer m that brings it into the window. Since the ruler is practically linear, that label points

$$\begin{aligned} \mathrm{posaux} &= c_w + x(f + m\mathrm{PRF}_{\mathrm{ch}}) \simeq x_t + m\,X, \\ X &= x(\mathrm{PRF}_{\mathrm{ch}}) = r\tan\Big(\arcsin\frac{\lambda\mathrm{PRF}_{\mathrm{ch}}}{2v}\Big)\frac{\mathrm{PRF}_{\mathrm{ch}}}{v} = 405.06\ \text{cells}, \end{aligned} \tag{7}$$

that is, m folds away from the scatterer (the nonlinearity of tan contributes less than 0.02 cells). With $|f| \le B/2 = 641\,\mathrm{Hz}$ the whole-band kernel sees $m \in \{-1, 0, +1\}$: at the beginning of the illumination ($f > \mathrm{PRF}_{\mathrm{ch}}/2$) the peak points to $x_t - X$, in the middle to x_t, at the end to $x_t + X$ (Fig. 4 and test T6). The map must therefore also hold the scatterer's residual at $x_t \pm X$, which Stage A does with the alias layers of Section 3.4.

2.5 The rotation

Each bin reads the map at its cell and is multiplied by

$$R_w(f) = e^{-j\,\mathrm{ph}(f)}, \qquad \mathrm{ph}(f) = -\frac{2\pi}{\lambda}\,\Delta C_0[\mathrm{posaux}(f)] \quad\Longrightarrow\quad R_w(f) = e^{+j2\pi\Delta C_0[\mathrm{posaux}(f)]/\lambda}, \tag{8}$$

which is the conjugate of the unwanted factor in Eq. (1) (`inverse=True` removes, `inverse=False` injects). If the map is constant, every bin of every window gets the same factor and Eq. (4) returns $s\,e^{+j2\pi\Delta C_0/\lambda}$ exactly (test T3). The complete kernel is

sata1d/sata.py, lines 51--85

```
51 def sata_1d(data, delta_C0_array, rref, prf, v, wl, r, squint=0.0, inverse=False,
       sata_osf=1):
52     """
53     data           : (naz,) complex azimuth line of one channel at prf (= PRF_op)
54     delta_C0_array : (naz,) residual range [m] per azimuth cell
55     inverse=True   : remove the residual (spec *= exp(-j ph)); False injects it.
56     """
57     data = np.asarray(data, dtype=complex)
58     dimx = len(data)
59     Tsubeff, hop, Nzp = subaperture_sizing(rref, prf, v, wl, sata_osf)
60     if Tsubeff <= 2:
61         return data.copy()
62     win = triangular_window(Tsubeff)
63 
64     fsub, azpos = subaperture_axis(prf, Nzp, v, wl, r, squint)
65 
66     out = np.zeros(dimx, dtype=complex)
67     wsum = np.zeros(dimx)
68     for start in range(0, dimx, hop):
69         seg = data[start:start + Tsubeff]
70         L = len(seg)
71         if L < 2:
72             break
73         buf = np.zeros(Nzp, dtype=complex)
74         buf[:L] = seg * win[:L]
75         spec = np.fft.fft(buf)
76         posaux = np.clip(np.round(azpos + start + 0.5 * Tsubeff).astype(int), 0, dimx - 1)
77         ph = -2.0 * np.pi / wl * delta_C0_array[posaux]
78         ph[~np.isfinite(ph)] = 0.0
79         spec *= np.exp(-1j * ph) if inverse else np.exp(1j * ph)
```

```
        out[start:start + L] += np.fft.ifft(spec)[:L]
        wsum[start:start + L] += win[:L]
    nz = wsum > 1e-12
    out[nz] /= wsum[nz]
    out[~nz] = data[~nz]
    return out
```

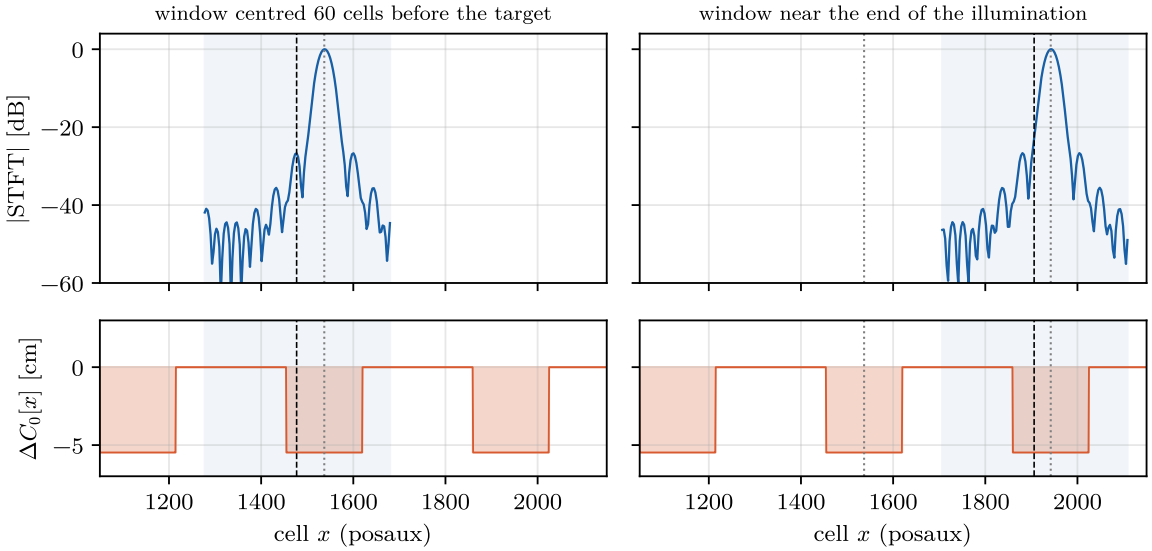


Figure 4: Two windows of channel 0 for the target at $h = 240\,\mathrm{m}$ (cell 1537). Top: |STFT| of the window, every bin drawn at its cell $c_w + x(f)$; the shaded band is the ±203 cells the window can reach; dashed: window centre; dotted: x_t and $x_t + X$. Bottom: the map $\Delta C_0[x]$ of Stage A. Left: the window is 60 cells before the target, the peak sits at x_t and reads −5.47 cm. Right: near the end of the illumination the true Doppler exceeds $+\mathrm{PRF}_{\mathrm{ch}}/2$, the peak is folded to $x_t + X = 1942$ and reads the alias block there, which carries the same value.

3 Stage A: the ΔC_0 Map

The kernel reads ΔC_0[posaux] for every bin, including bins that hold no energy. A bin's rotation matters only in proportion to its energy, so the map has to be right where energy appears and can be anything elsewhere. The implementation writes each scatterer's residual on the cells where its energy can land and zero elsewhere. Figure 5 shows the five steps.

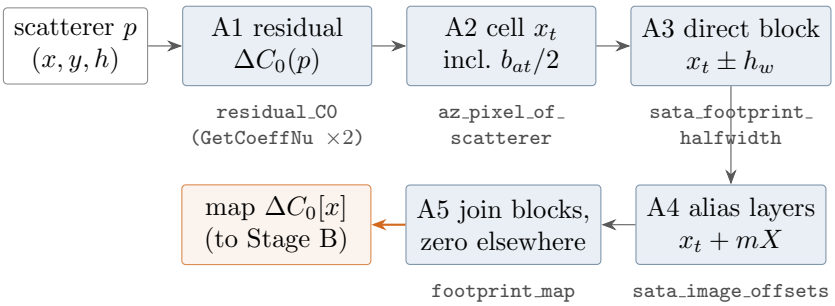


Figure 5: Stage A, `build_delta_C0_array`, once per channel (and, per sub-band, once per channel and band).

3.1 A1 — the residual of each scatterer

The residual is evaluated with the same semi-numerical model that builds the filters, so that error and correction are consistent by construction (report, Eq. 36). `GetCoeffNu` fits two parabolas over the common illuminated aperture: the monostatic history $R_{ms}(t) = 2R_{Tx}(t) \simeq c_0+c_1t+c_2t^2$ and the bistatic-minus-monostatic difference $R_{bs}(t)-R_{ms}(t) \simeq \Delta c_0+\Delta c_1 t+\Delta c_2 t^2$. The value of a parabola at its vertex is $c_0 - c_1^2/4c_2$, and C_0 is the difference of the two vertex values:

$$C_0 = \left[(c_0+\Delta c_0) - \frac{(c_1+\Delta c_1)^2}{4(c_2+\Delta c_2)}\right] - \left[c_0 - \frac{c_1^2}{4c_2}\right] = \Delta c_0 + \frac{c_1^2}{4c_2} - \frac{(c_1+\Delta c_1)^2}{4(c_2+\Delta c_2)}. \tag{9}$$

`residual_C0` calls it twice, for the scatterer and for the reference point, and returns the difference, Eq. (1):

sata1d/sata.py, lines 91--96

```
91 def residual_C0(cfg: ExperimentConfig, tracks: PlatformTracks, ptg_real, channel):
92     """C0(scatterer) - C0(reference point) for one channel [m]."""
93     common = (tracks.ptx, tracks.prx[channel], tracks.vtx, tracks.vrx[channel],
94               tracks.ptx, tracks.vtx, cfg.prf, cfg.system.wl, cfg.ta,
95               cfg.sq_tx, cfg.sq_rx[channel], cfg.theta_tx, cfg.theta_rx[channel])
96     return float(GetCoeffNu(ptg_real, *common)[0] - GetCoeffNu(cfg.scene.ptg, *common)[0])
```

For the target at $h = 240\,\mathrm{m}$ this gives $\Delta C_0 = [-5.474,\ -2.319,\ -0.352,\ -0.142]\,\mathrm{cm}$ for the four channels, within 0.3 % of Eq. (2) (test T4, Fig. 11).

3.2 A2 — where the scatterer sits on the channel line

The channel line is indexed by azimuth cells of $d_s = v/\mathrm{PRF_{ch}}$, with the scene centre at $N_{a,ch}/2$. The Tx–Rx phase centre of channel i trails the transmitter by $b_{at,i}/2$, so the closest approach of scatterer $x_0 + \Delta x$ falls at

$$x_t = \left\lfloor \frac{N_{a,ch}}{2} + \frac{x_0 + \Delta x + b_{at,i}/2}{d_s} \right\rceil. \tag{10}$$

For the DPCA array ($b_{at}/2 = 0,\ 3.84,\ 7.69,\ 11.53\,\mathrm{m}$) this is cell 1537 for channel 0 and 1538 for the others, identical to the minimum of the simulated range history (test T5).

sata1d/sata.py, lines 99--103

```
99 def az_pixel_of_scatterer(cfg: ExperimentConfig, dx, channel):
100     """Cell of scatterer x0 + dx on the per-channel line; the Tx-Rx phase centre
101     trails the transmitter by bat/2."""
102     ds = cfg.system.vs / cfg.PRF_op
103     return int(round(cfg.Na_ch / 2 + (cfg.scene.x0 + dx + 0.5 * cfg.array.bat[channel]) /
        ds))
```

3.3 A3 — the footprint: how far a scatterer's energy spreads

Within one window the scatterer is a nearly pure tone: its Doppler changes by $K_a\,T_{\mathrm{sub,eff}}/\mathrm{PRF_{ch}} = 617\,\mathrm{Hz/s} \cdot 0.04\,\mathrm{s} \approx 25\,\mathrm{Hz}$. Its spectrum is therefore the transform of the triangular window, $|W(f)| \propto \mathrm{sinc}^2(f\,T_{\mathrm{sub,eff}}/2\mathrm{PRF_{ch}})$, centred on its instantaneous Doppler, with nulls at multiples of $2\mathrm{PRF_{ch}}/T_{\mathrm{sub,eff}} = 50\,\mathrm{Hz}$. The map covers the response up to the second null,

$$f_{\mathrm{edge}} = \frac{4\,\mathrm{PRF_{ch}}}{T_{\mathrm{sub,eff}}} = 100\ \mathrm{Hz}, \qquad h_w = \lceil x(f_{\mathrm{edge}}) \rceil = 82\ \mathrm{cells}\ (\approx 1.26\ \mathrm{km}), \tag{11}$$

and every bin that holds energy of the scatterer then points inside $x_t \pm h_w$. The first null (41 cells) left 4.8 % error on the corrected channel; the second null leaves 2 % and covers 99.9 % of the energy (tests T8, T9).

sata1d/sata.py, lines 106--115

```
106 def sata_footprint_halfwidth(cfg: ExperimentConfig, squint=0.0):
107     """Half-width [cells] of one scatterer in a sub-aperture spectrum: triangular
108     window up to its 2nd null (4 PRF_op / Tsubeff), mapped through the ruler."""
109     prf, v, wl, r = cfg.PRF_op, cfg.system.vs, cfg.system.wl, cfg.scene.r0
110     Tsubeff, _, _ = subaperture_sizing(r, prf, v, wl)
111     f_edge = 4.0 * prf / Tsubeff
112     fc = 2 * v / wl * np.sin(squint)
113     b0 = np.arcsin(np.clip(wl * fc / (2 * v), -1, 1))
114     b1 = np.arcsin(np.clip(wl * (fc + f_edge) / (2 * v), -1, 1))
115     return int(np.ceil(r * (np.tan(b1) - np.tan(b0)) / v * prf))
```

3.4 A4 — alias layers

From Section 2.4, the same energy also lands at $x_t + mX$. For a kernel whose labels cover $[f_c - \mathrm{PRF}_{\mathrm{ch}}/2,\ f_c + \mathrm{PRF}_{\mathrm{ch}}/2)$ the folds that occur are

$$m \in \left\{ \left\lfloor \frac{f_c - B/2}{\mathrm{PRF}_{\mathrm{ch}}} \right\rceil, \ \ldots, \ \left\lfloor \frac{f_c + B/2}{\mathrm{PRF}_{\mathrm{ch}}} \right\rceil \right\}, \tag{12}$$

which gives $\{-1, 0, 1\}$ for the whole band ($f_c = 0$) and one-sided sets for the sub-bands ($f_c = f_k$, Section 4).

sata1d/sata.py, lines 118--129

```
118 def sata_image_offsets(cfg: ExperimentConfig, f_centre=0.0):
119     """Offsets [cells] where one scatterer appears for a kernel whose bins are labelled
120     with the absolute Doppler in [f_centre - PRF_op/2, f_centre + PRF_op/2).
121     A true Doppler f (|f| <= B/2, B = 4 v sin(theta/2)/wl) is labelled f + m PRF_op,
122     so it lands m*X cells away, X = cell shift of one PRF_op fold.
123     Whole band (f_centre = 0): {-X, 0, +X}; sub-band k: one-sided set."""
124     prf, v, wl, r = cfg.PRF_op, cfg.system.vs, cfg.system.wl, cfg.scene.r0
125     X = r * np.tan(np.arcsin(np.clip(wl * prf / (2 * v), -1, 1))) / v * prf
126     B = 4.0 * v * np.sin(cfg.theta_tx / 2.0) / wl
127     m_lo = int(np.round((f_centre - B / 2) / prf))
128     m_hi = int(np.round((f_centre + B / 2) / prf))
129     return np.array([m * X for m in range(m_lo, m_hi + 1)], dtype=float)
```

3.5 A5 — drawing the map

`footprint_map` writes three layers in priority order (Fig. 6):

1. *Direct layer.* Scatterers whose footprints touch (gap $\leq 2h_w$) form one terrain block, linearly interpolated between their values over $[x_{\mathrm{first}} - h_w,\ x_{\mathrm{last}} + h_w]$; an isolated scatterer gives a constant block $x_t \pm h_w$.
2. *Alias layers.* The direct layer shifted by each $mX \neq 0$, written only on cells the direct layer leaves empty.
3. *Zero* on every other cell. Those bins hold no scatterer energy.

sata1d/sata.py, lines 132--169

```
132 def footprint_map(values_by_pixel: dict, naz: int, halfwidth: int, offsets=(0.0,)):
133     """
```

```
    1) direct layer: scatterers whose footprints touch (gap <= 2 hw) form one block,
       linearly interpolated between them over [first - hw, last + hw];
       an isolated scatterer gives a constant block pixel +- hw;
    2) alias layers: the direct layer shifted by each non-zero offset, written only
       where the direct layer is empty;
    3) zero elsewhere.
    """
    arr = np.zeros(naz)
    if not values_by_pixel:
        return arr
    pix = np.array(sorted(values_by_pixel), dtype=float)
    val = np.array([max(values_by_pixel[p], key=abs) for p in sorted(values_by_pixel)])
    x = np.arange(naz)
    cov = np.zeros(naz, bool)
    cuts = np.flatnonzero(np.diff(pix) > 2 * halfwidth) + 1
    for bp, bv in zip(np.split(pix, cuts), np.split(val, cuts)):
        lo = int(max(0, np.floor(bp[0] - halfwidth)))
        hi = int(min(naz - 1, np.ceil(bp[-1] + halfwidth)))
        if lo > hi:
            continue
        arr[lo:hi + 1] = np.interp(x[lo:hi + 1], bp, bv)
        cov[lo:hi + 1] = True
    direct, dcov = arr.copy(), cov.copy()
    for off in sorted(offsets, key=abs):
        sh = int(round(off))
        if sh == 0:
            continue
        src, scov = np.roll(direct, sh), np.roll(dcov, sh)
        if sh > 0:
            scov[:sh] = False
        else:
            scov[sh:] = False
        put = scov & ~cov
        arr[put] = src[put]
        cov |= put
    return arr
```

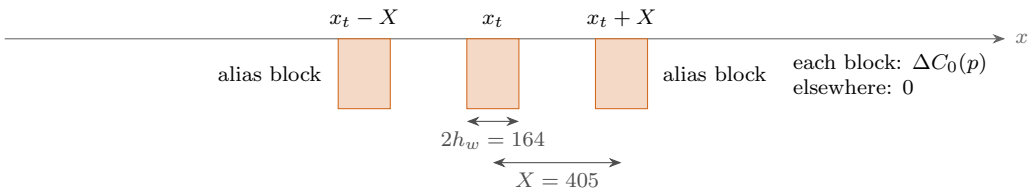


Figure 6: Map of one isolated scatterer for the whole-band kernel (channel 0: $x_t = 1537$, value -5.47 cm). The direct block receives the peak in the central part of the illumination, the alias blocks the peaks at the two ends (Fig. 4).

The rule illustrated: two scatterers with overlapping apertures. Where the illuminated spans of two scatterers overlap, a window sees both. Its spectrum then has two peaks, one per scatterer, and by Section 2.3 each peak points at its own scatterer's cell. Each peak therefore reads its own block of the map and is rotated with its own residual (Fig. 7). This is what a single phase per window or per line cannot do. The previous map, which held the value of the nearest scatterer over the whole line (`mode="hold"`), gave the right phase to one of the two peaks only.

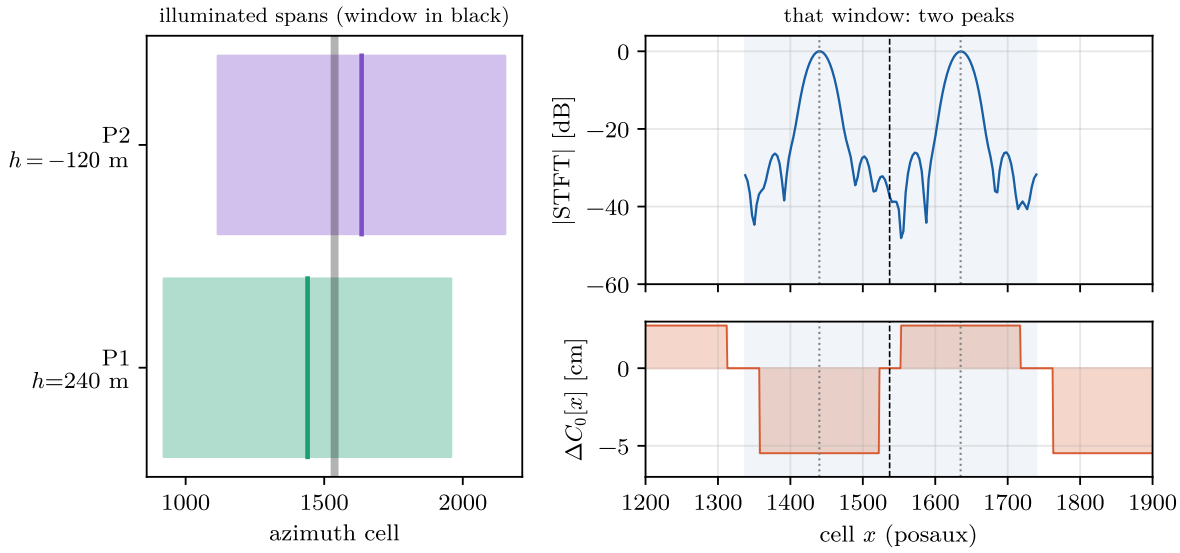


Figure 7: P1 ($h = 240\,\mathrm{m}$, $\Delta C_0 = -5.47\,\mathrm{cm}$) and P2 ($h = -120\,\mathrm{m}$, $\Delta C_0 = +2.74\,\mathrm{cm}$), 3 km apart, channel 0. Left: their illuminated spans overlap; the window marked in black sees both. Right: the spectrum of that window has two peaks, one at each scatterer's cell (dotted), and each falls in its own block of the map. The blocks at the edges are the alias layers of the other scatterer ($x_2 - X$, $x_1 + X$).

The resolution limit. The spectrum of a window separates two scatterers only if their peaks are more than about one resolution cell apart, that is, if the scatterers are more than $\delta x \simeq 309\,\mathrm{m}$ apart (20 cells). Closer scatterers share bins, and the joined block interpolates between their residuals. This is exact for a smooth slope and approximate for a jump. It is the fundamental limit of the method, not of the implementation (the "spike" scene in Section 5).

4 Whole Band and per Sub-Band

4.1 What changes

The reconstruction delivers N_{rx} Doppler sub-bands; sub-band k is centred at (report, Eq. 46)

$$f_k = \Big(-\frac{N_{rx}}{2} + k + \frac{1}{2} \Big) \mathrm{PRF}_{\mathrm{ch}} = -750,\ -250,\ +250,\ +750\ \mathrm{Hz}, \qquad \beta_k = \arcsin \frac{\lambda f_k}{2v}. \tag{13}$$

The *whole-band* variant corrects each channel once and feeds that single set to all sub-bands. The *per-sub-band* variant repeats both stages for every k (Fig. 8):

- A1 fits ΔC_0 only over the look angles of the band, $[\beta(f_k - \mathrm{PRF}_{\mathrm{ch}}/2),\ \beta(f_k + \mathrm{PRF}_{\mathrm{ch}}/2)]$ (`getcoeff_beam`);
- the kernel labels its bins with the absolute Doppler of band k (below);
- A4 uses the fold set of Eq. (12) with $f_c = f_k$;
- sub-band k of the reconstruction uses only the set corrected for k.

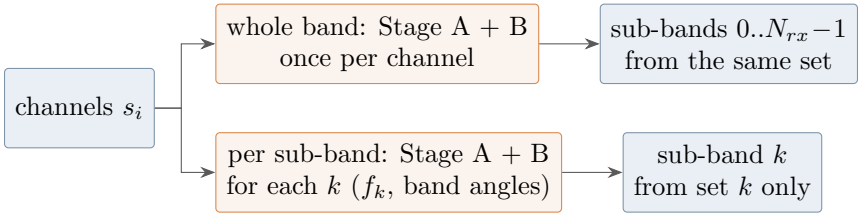


Figure 8: The two variants. The per-sub-band one costs N_{rx} corrections and N_{rx} reconstructions instead of one of each.

4.2 The sub-band axis

The kernel of band k must label every bin with the absolute Doppler it holds within $[f_k - \mathrm{PRF}_{\mathrm{ch}}/2,\ f_k + \mathrm{PRF}_{\mathrm{ch}}/2)$ and measure positions on the broadside grid on which the map is defined (report, Section 4.2, points 2 and 3). A baseband bin ϕ (`fftfreq`) holds the true Dopplers $\phi + m\mathrm{PRF}_{\mathrm{ch}}$, and exactly one of them lies in the window of band k:

$$\begin{aligned} f_{\mathrm{lo}} &= f_k - \frac{\mathrm{PRF}_{\mathrm{ch}}}{2}, \qquad \tilde{f}(\phi) = f_{\mathrm{lo}} + \left[(\phi - f_{\mathrm{lo}}) \bmod \mathrm{PRF}_{\mathrm{ch}}\right], \\ x(\tilde{f}) &= r \tan\beta(\tilde{f})\, \frac{\mathrm{PRF}_{\mathrm{ch}}}{v} \qquad (\beta_{\mathrm{img}} = 0). \end{aligned} \tag{14}$$

Energy of band k then points straight at x_t, while energy of the other bands points at $x_t + mX$ with m from Eq. (12). Figure 9 shows how the map moves its alias layers with k.

sata1d/subband.py, lines 94--101

```
94 def subband_axis(prf, Nzp, v, wl, r, f_k):
95     """Absolute Doppler of each FFT bin for sub-band k (baseband frequency wrapped into
96     [f_k - prf/2, f_k + prf/2)) and its cell offset on the broadside grid."""
97     phi = np.fft.fftfreq(Nzp, d=1.0 / prf)
98     f_lo = f_k - prf / 2.0
99     fsub = f_lo + np.mod(phi - f_lo, prf)
100    betasub = np.arcsin(np.clip(wl * fsub / (2.0 * v), -1.0, 1.0))
101    return fsub, r * np.tan(betasub) / v * prf
```

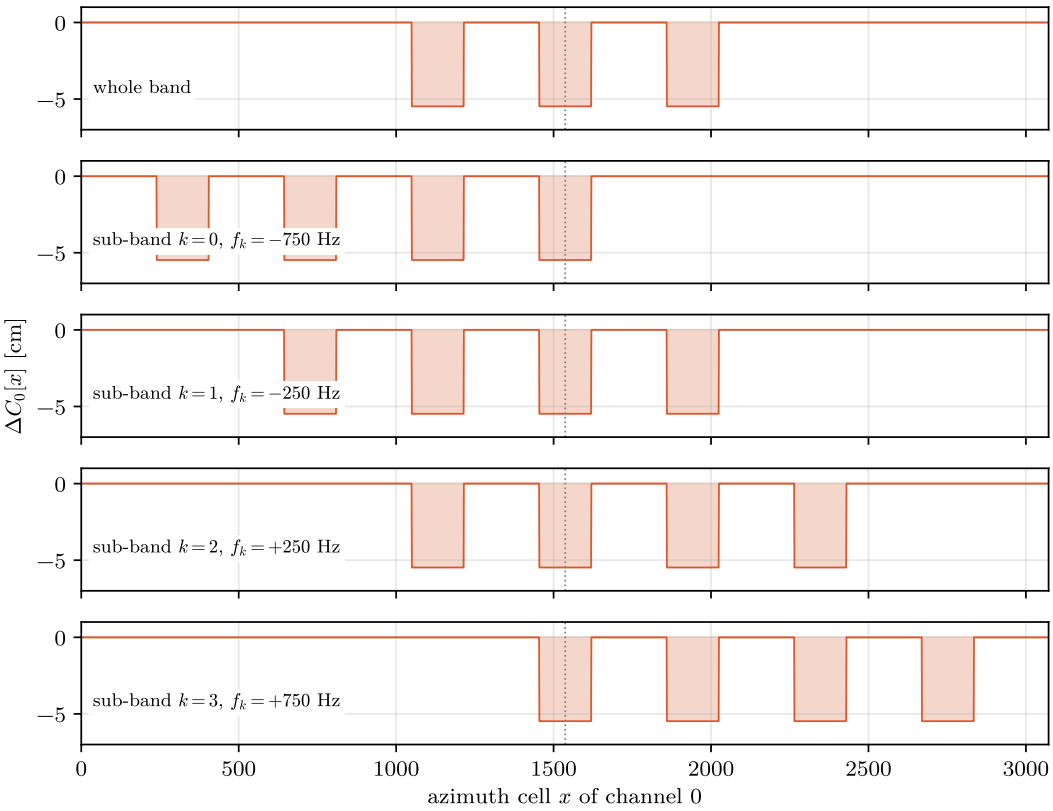


Figure 9: Maps of channel 0 for the whole band and the four sub-bands. The direct block at $x_t = 1537$ is always present; the alias layers follow the fold set of each band: $\{-1, 0, 1\}$ for the whole band and $\{-3..0\}$, $\{-2..1\}$, $\{-1..2\}$, $\{0..3\}$ for $k = 0..3$. The value is the same in all bands because ΔC_0 of an iso-range target does not depend on the look angle.

4.3 Correction of the previous per-sub-band axis

The previous one-dimensional per-sub-band kernel used $f = f_k + \phi$ and positions measured from the squint of the band, $x = r[\tan\beta(f_k+\phi) - \tan\beta_k]\,\mathrm{PRF}_{\mathrm{ch}}/v$. Both terms are inconsistent with Eq. (14). Since f_k is an odd multiple of $\mathrm{PRF}_{\mathrm{ch}}/2$, the label $f_k+\phi$ is not one of the Dopplers $\phi + m\mathrm{PRF}_{\mathrm{ch}}$ that bin ϕ actually holds. Subtracting $\tan\beta_k$ then shifts the grid back by the same half band. To first order the two errors cancel, $x \simeq 0.81\,\phi$, and the "per-sub-band" kernel was position-for-position the whole-band kernel. The energy of band k did not point at x_t but at x_t, $x_t \pm X$ depending on the fold (Fig. 10, left; test T7: up to 407 cells, 60 % of the windows off by at least one fold). It still read the right value only because its map had the whole-band alias set. This is the "axis of the right width arising from two compensating errors" described in the report. With Eq. (14) every window of band k points at the target to within one bin (right panel, max. 2.4 cells).

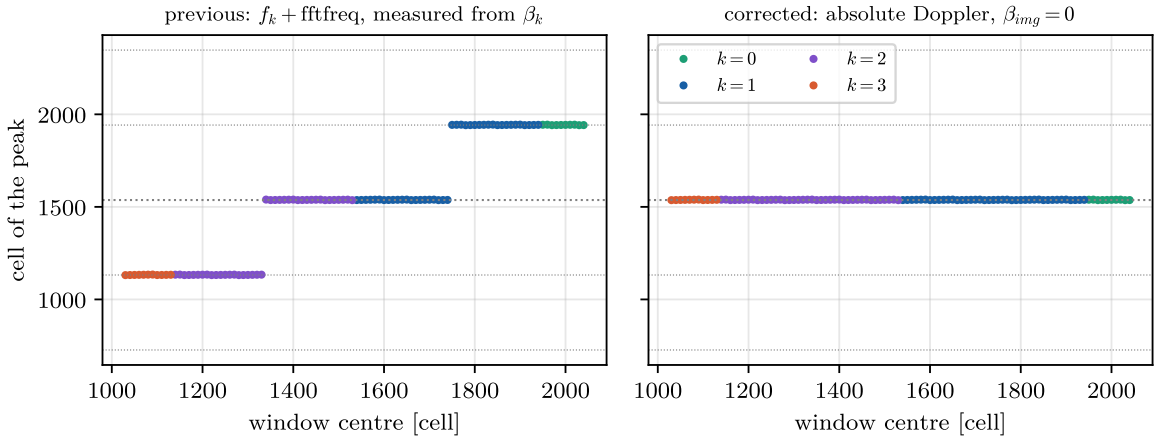


Figure 10: Cell pointed to by the STFT peak, for the windows whose instantaneous Doppler lies in band k (colours). Dotted: x_t and $x_t + mX$. Left: previous axis. Right: corrected axis of Eq. (14).

The kernel of band k is `sata_1d` with that axis:

sata1d/subband.py, lines 104--134

```
104 def sata_1d_subband(data, delta_C0_array, rref, prf, v, wl, r, f_k, inverse=False,
        sata_osf=1):
105     """sata_1d for sub-band k: bins labelled with their absolute Doppler in the
106     sub-band window (subband_axis), positions on the broadside grid."""
107     data = np.asarray(data, dtype=complex)
108     dimx = len(data)
109     Tsubeff, hop, Nzp = subaperture_sizing(rref, prf, v, wl, sata_osf)
110     if Tsubeff <= 2:
111         return data.copy()
112     win = triangular_window(Tsubeff)
113     fsub, azpos = subband_axis(prf, Nzp, v, wl, r, f_k)
114
115     out = np.zeros(dimx, dtype=complex)
116     wsum = np.zeros(dimx)
117     for start in range(0, dimx, hop):
118         seg = data[start:start + Tsubeff]
119         L = len(seg)
120         if L < 2:
121             break
122         buf = np.zeros(Nzp, dtype=complex)
123         buf[:L] = seg * win[:L]
124         spec = np.fft.fft(buf)
125         posaux = np.clip(np.round(azpos + start + 0.5 * Tsubeff).astype(int), 0, dimx - 1)
126         ph = -2.0 * np.pi / wl * delta_C0_array[posaux]
127         ph[~np.isfinite(ph)] = 0.0
128         spec *= np.exp(-1j * ph) if inverse else np.exp(1j * ph)
129         out[start:start + L] += np.fft.ifft(spec)[:L]
130         wsum[start:start + L] += win[:L]
131     nz = wsum > 1e-12
132     out[nz] /= wsum[nz]
133     out[~nz] = data[~nz]
134     return out
```

and the per-sub-band reconstruction assembles the N_{rx} corrected sets:

sata1d/subband.py, lines 137--158

```
137 def sata_channels_subband(cfg, tracks, s_channel, k, remove=True, sata_osf=4,
        mode="footprint"):
```

```
    """Every channel SATA-corrected for output sub-band k."""
    f_k = subband_frequency_beam(cfg, k)[0]
    out = np.asarray(s_channel, dtype=complex).copy()
    for kk in range(cfg.Nrx):
        dC0 = build_delta_C0_subband_array(cfg, tracks, kk, k, mode=mode)
        out[kk] = sata_1d_subband(out[kk], dC0, rref=cfg.scene.r0, prf=cfg.PRF_op,
    v=cfg.system.vs,
                                  wl=cfg.system.wl, r=cfg.scene.r0, f_k=f_k,
                                  inverse=remove, sata_osf=sata_osf)
    return out


def reconstruct_subband(cfg: ExperimentConfig, tracks: PlatformTracks, s_channel,
    use_sata=True,
                        remove=True, sata_osf=4, mode="footprint", zeroOutBw=True):
    """Reconstruction where sub-band k uses the channels corrected for sub-band k.
    use_sata=False gives the standard reconstruction."""
    base = np.asarray(s_channel, dtype=complex)
    spec = []
    for k in range(cfg.Nrx):
        corr = sata_channels_subband(cfg, tracks, base, k, remove, sata_osf, mode) if
    use_sata else base
        spec.append([channel_spectrum(cfg, corr[j]) for j in range(cfg.Nrx)])
    return reconstruct_from_spectra(cfg, tracks, spec, zeroOutBw)
```

5 Verification

`verify_sata1d.py` runs eleven tests on the reference configuration (Table 3 of the report, $N_{rx} = 4$, DPCA along-track baselines, $b_{xt} \sim U(0, 100)$ m, seed 0) and writes a report and figures to `verification/`. Table 1 lists what each test checks, its criterion and the result.

Table 1: Verification tests (`python verify_sata1d.py`).

	What is checked	Criterion	Result	
T1	zero map returns the input (whole band + 4 sub-bands)	rel. err. $< 10^{-12}$	1.2×10^{-16}	pass
T2	inject then remove the pipeline maps	rel. err. $< 2\,\%$	$0.79\,\%$	pass
T3	constant map = one global rotation $e^{+j2\pi\Delta C_0/\lambda}$	rel. err. $< 10^{-12}$	1.4×10^{-16}	pass
T4	`residual_C0` vs closed form, 4 heights $\times$ 4 channels	ratio within $2\,\%$	0.9976–0.9998	pass
T5	cell of Eq. (10) vs simulated closest approach	≤ 1 cell	0 cells	pass
T6	whole band: STFT peak $\to x_t + mX$ (102 windows)	≤ 1 bin (3.2 cells)	2.4 cells	pass
T7	per sub-band: own-band peak $\to x_t$	≤ 1 bin	2.4 cells	pass
T8	fraction of the target's STFT energy read inside the map	$\geq 99\,\%$	$99.91\,\%$ (all 5)	pass
T9	corrected channel vs $e^{+j2\pi\Delta C_0/\lambda}\,s$, 4 ch. $\times$ 5	$< 5\,\%$	$1.97\,\%$	pass
T10	ambiguity after reconstruction, $h = 0..400$ m	within 1 dB of ref.	0.27 dB	pass
T11	multi-target scenes vs oracle	reported	Table 2	—

Kernel algebra (T1–T3). T1 and T3 hold to machine precision, as Eq. (4) predicts. T2 holds to $0.8\,\%$ for the maps the pipeline produces. Injection and removal are not exact inverses of each other, because windows overlap and the re-analysed spectra mix neighbouring bins. The error grows with how fast the phase varies across one sub-aperture response: a white-noise map (± 30 cm per cell) comes back with $96\,\%$ error. This is inherent to SATA: the correction can only represent a residual that is slowly varying on the scale of δx.

Residual and position (T4, T5). Figure 11 compares the numerical residual with Eq. (2).

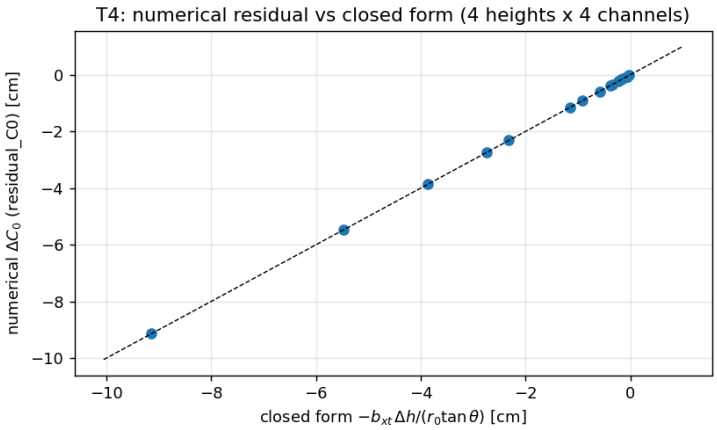


Figure 11: T4: numerical ΔC_0 against the closed form, four heights (40–400 m) and four channels.

Ruler (T6, T7). Figure 12 tracks the peak of every window across the illumination: $x_t - X$ at the start, x_t in the middle, $x_t + X$ at the end, with at most 2.4 cells of scatter, which is below the 3.2-cell width of one bin. This verifies Eqs. (6) and (7) on the simulated data and justifies the alias layers.

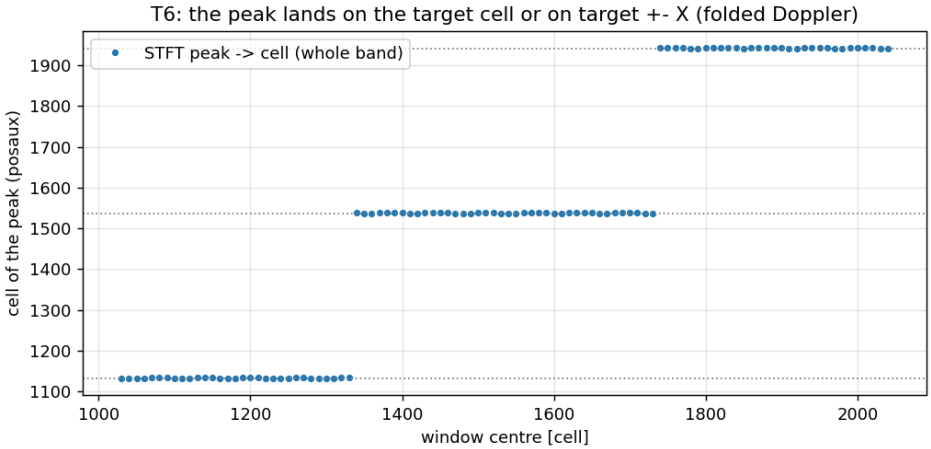


Figure 12: T6: cell of the STFT peak against the window centre, channel 0, whole-band kernel.

Correction and reconstruction (T8–T10). Of the target's STFT energy, 99.91 % falls on non-zero cells of the map, and the corrected channel is within 2 % of the ideal rotation. The remaining 2 % is energy in the sidelobes beyond $\pm h_w$, which reads zero. After reconstruction the worst ambiguity is −39.4 dB for all heights, against −39.1 dB for the reference (the scene reconstructed with the true height in the filters) and −22.9 to −9.0 dB without correction (Figs. 13 and 14). Whole band and per sub-band coincide, as expected for a single iso-range target, whose ΔC_0 is the same at every look angle (Fig. 9).

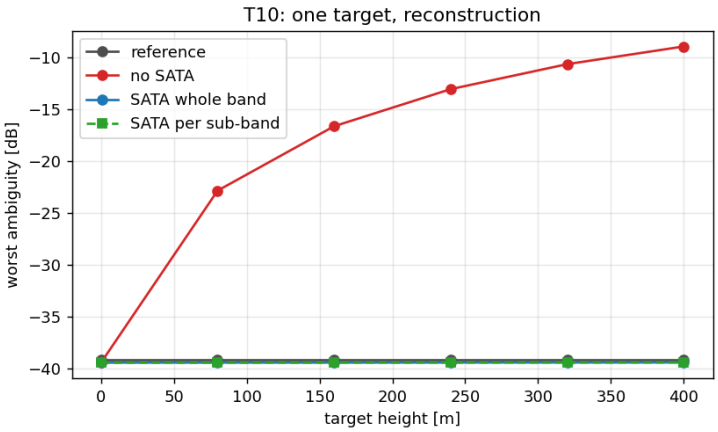


Figure 13: T10: worst azimuth ambiguity against target height.

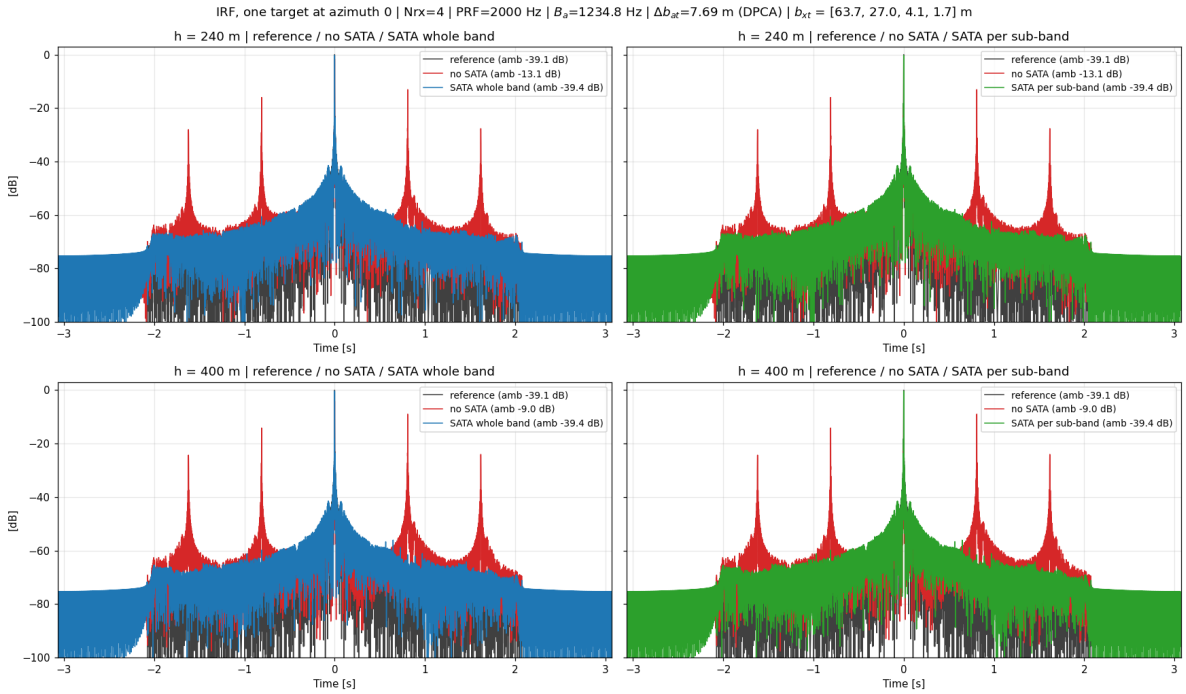


Figure 14: Impulse response of one target, $h = 240\,\mathrm{m}$ (top) and $400\,\mathrm{m}$ (bottom); left whole band, right per sub-band (`plot_irf_single_target.py`). The ambiguous replicas at multiples of $1/\mathrm{PRF}_{\mathrm{ch}}$ disappear with SATA. The broad floor between -75 and $-60\,\mathrm{dB}$ is the 2 % residual of T9 spread by the azimuth compression; it lies well below the ambiguity level.

Multi-target scenes (T11). The benchmark for a scene with several scatterers is the *oracle*: each scatterer's signal is corrected with its own ΔC_0 before the channels are summed, which is what a perfect position-selective correction would achieve. Table 2 and Fig. 15 compare it with SATA (whole band and per sub-band), with the previous map, and with no correction.

Table 2: T11: worst azimuth ambiguity [dB] (lower is better).

scene	ref.	oracle	whole	sub-band	prev. map	no SATA
2 targets 3 km apart (240 / −120 m)	−37.9	−38.3	−38.3	−38.3	−15.1	−12.8
3 targets 3 km apart (0 / 300 / 0 m)	−38.4	−38.5	−38.7	−38.6	−16.8	−11.2
ramp 5 × 200 m, 3°	−34.3	−31.0	−31.0	−31.0	−31.0	−28.6
ramp 9 × 200 m, 15°	−33.9	−30.8	−30.8	−30.8	−19.3	−8.6
random heights 5 × 200 m (0–100 m)	−34.3	−31.0	−31.1	−31.1	−26.6	−21.3
cliff 5 × 200 m, 100 m jump	−34.3	−31.0	−31.1	−31.1	−25.7	−20.8
spike 5 × 200 m, 500 m	−34.3	−31.0	−13.1	−13.1	−10.2	−7.6



Figure 15: T11 as bars. Where SATA can separate the scatterers it reaches the oracle.

Three observations follow. First, SATA reaches the oracle to within 0.3 dB in six of the seven scenes, including the two scenes with scatterers 3 km apart. There the previous map loses more than 20 dB, because it gives the residual of one scatterer to the other's peaks (Fig. 7). Second, on the 200-m scenes the oracle itself stays about 3 dB above the reference. That gap is not SATA's: the oracle removes ΔC_0 exactly, and what is left comes from the linear and quadratic residuals ΔC_1, ΔC_2, which this correction does not address. Third, the spike scene is the resolution limit of Section 3: a 500 m jump between scatterers 200 m apart ($< \delta x$) cannot be separated in the sub-aperture spectra, and the interpolated block is wrong for both.

Common residual. With $b_{xt} = 0$ every channel carries the same residual, and `run_geometry_sweep.py --quick` confirms that the correction then changes nothing (all variants identical to no correction): a phase common to all channels costs nothing (report, Eq. 40), so there is nothing to correct.

5.1 Full Geometry Sweep: Scenes with Several Targets

The full sweep repeats the case matrix of the report with the current code (sub-band axis included): 181 scenes in nine families of height profiles, 2 to 13 point targets 200 m apart on the iso-range surface, $b_{xt} \sim U(0, 100)$ m. Every scene is reconstructed five times: reference (filters told the true heights), no correction, SATA whole band, SATA per sub-band, and the previous map (`mode="hold"`) in both variants.

What the spacing implies. The spacing of 200 m is below the SATA resolution $\delta x \simeq 309$ m and far below $2h_w$ (164 cells, 2.5 km). All footprints therefore touch and every scene becomes a single terrain block in which the map interpolates linearly between the targets (Section 3, A5). The sweep is thus a test of that interpolation: it is exact where the profile is smooth on the scale of δx and fails where neighbouring targets differ strongly in height. The best any ΔC_0 correction can do on these scenes is the oracle, about 3.3 dB above the reference because of the uncorrected ΔC_1, ΔC_2 (Table 2).

Table 3: Full sweep, $b_{xt}^{\max} = 100$ m: median worst ambiguity [dB] per family (181 scenes).

family	scenes	ref.	no SATA	whole	sub-band	prev. (whole)	prev. (sub)
ramp	78	−34.5	−22.5	−31.0	−31.0	−28.2	−23.3
zigzag	26	−34.1	−30.8	−30.8	−30.8	−30.8	−30.8
piecewise	15	−33.9	−14.1	−28.3	−28.2	−19.7	−15.7
steepening	8	−34.1	−12.8	−30.8	−30.8	−21.0	−15.7
flattening	8	−34.1	−12.7	−30.8	−30.8	−20.3	−15.7
cliff	14	−34.1	−12.7	−25.6	−25.6	−19.8	−17.2
spike	8	−34.1	−8.3	−11.9	−11.9	−10.0	−8.4
mesa	6	−34.1	−8.0	−18.7	−18.7	−11.3	−8.2
random heights	18	−33.9	−7.9	−17.8	−17.8	−15.4	−13.4

Overall result. Against the previous map the new one is better by more than 0.5 dB in 125 of the 181 scenes, equal in 53 and worse in 3; the median improvement is 3.7 dB (Fig. 16). Whole band and per sub-band differ by at most 0.07 dB in every scene, so the sub-band axis correction changes nothing for these iso-range targets, as expected (Section 4). The previous map was already worse per sub-band than whole band, because its held values are read at the folded positions of each band.

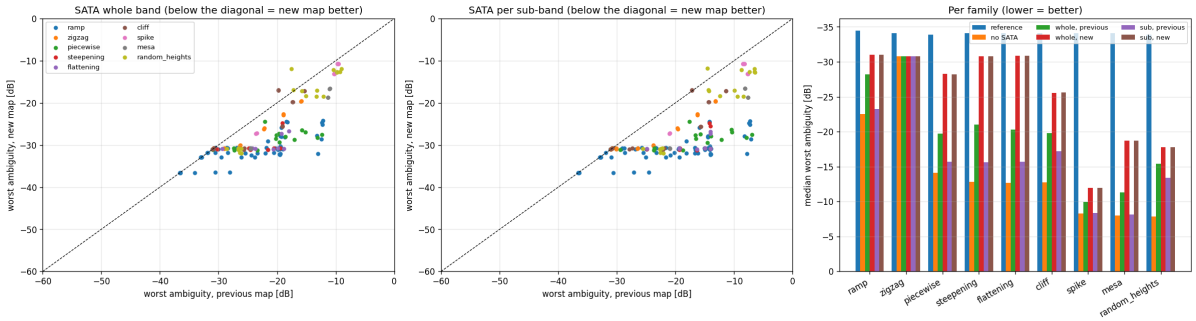


Figure 16: Full sweep. Left and centre: worst ambiguity with the new map against the previous one, one dot per scene; below the diagonal the new map is better. Right: median per family for the six reconstructions.

What decides success: the largest jump between neighbours. Figure 17 orders all scenes by the largest height difference between two neighbouring targets. With the new map every scene with jumps up to about 100 m stays at the oracle level (3.3 dB above the reference, dotted), whatever the family, the number of targets or the total height range; the error grows only beyond that, where two targets 200 m apart differ so much that one interpolated value cannot serve both. The previous map degrades already from jumps of about 10 m, because it assigns one held value to bins that see different targets. For channel 0 a 100 m jump is a residual difference of 2.3 cm, i.e. 33° of phase between neighbours closer than δx.

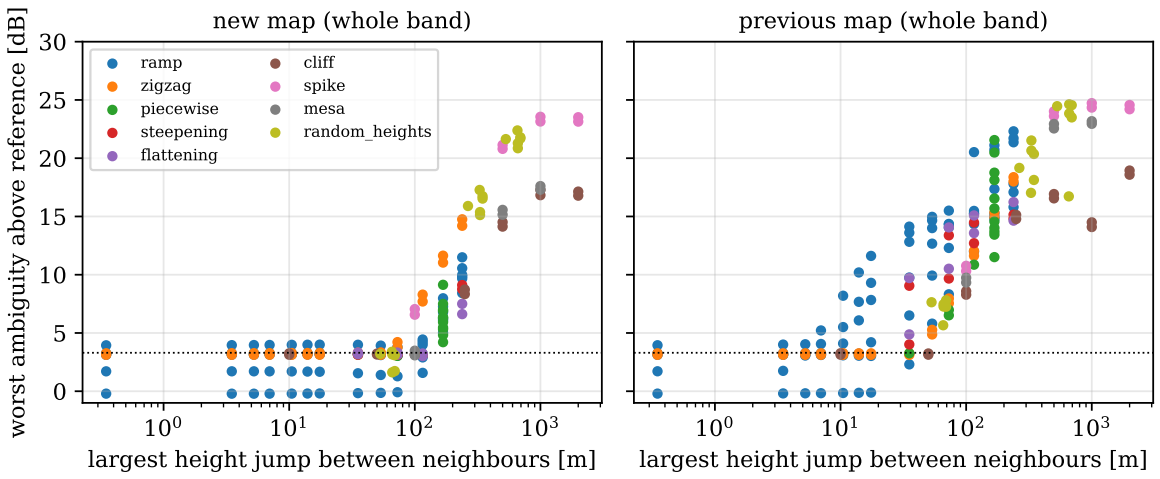


Figure 17: Excess ambiguity over the reference against the largest neighbour-to-neighbour height jump (log scale), all 181 scenes, whole band. Dotted: the oracle level of the 200-m scenes. Left: new map. Right: previous map.

Improvement over no correction. Figures 18 and 19 give, scene by scene, the gain of SATA over the uncorrected reconstruction, worst ambiguity without SATA minus worst ambiguity with SATA (positive: SATA better). For the uniform ramps the gain grows with the inclination and with the number of targets (a longer ramp reaches larger heights) up to 26 dB, median 10.1 dB, against 5.8 dB for the previous map. It falls again at 40–50°, where neighbours are 170–240 m apart in height, beyond the limit of Fig. 17. A gain near zero does not mean a failure: in all 36 ramp and zigzag scenes with less than 0.5 dB of gain the uncorrected reconstruction is already within 4 dB of the reference, i.e. at the oracle level, and there is nothing left for a ΔC_0 correction to remove. Among the other families, the smooth profiles (steepening, flattening, piecewise) gain 7–23 dB (one piecewise scene, PW-004, only −12 to +30 m in height, is already at the oracle level without correction), while spike, mesa and random heights gain 2–11 dB, limited by their large jumps. No scene gets worse with SATA (the smallest gain is −0.03 dB). The per-sub-band maps are identical to within 0.1 dB and are not repeated.

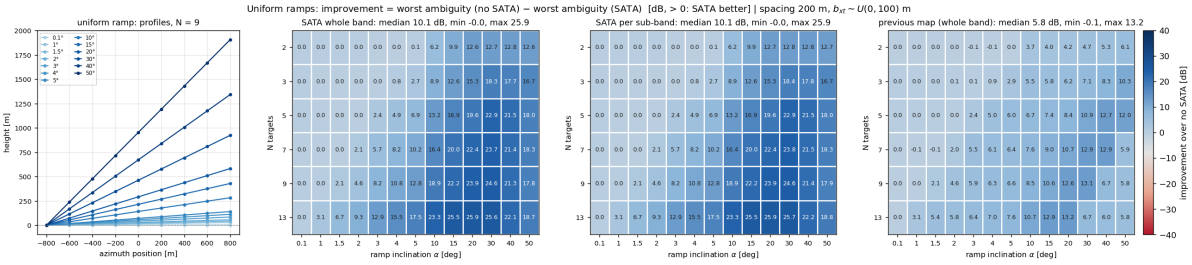


Figure 18: Uniform ramps. Left: the height profiles for $N = 9$, one curve per inclination. Right: improvement over no correction [dB] for N targets (rows) and inclination α (columns); SATA whole band, SATA per sub-band and the previous map.

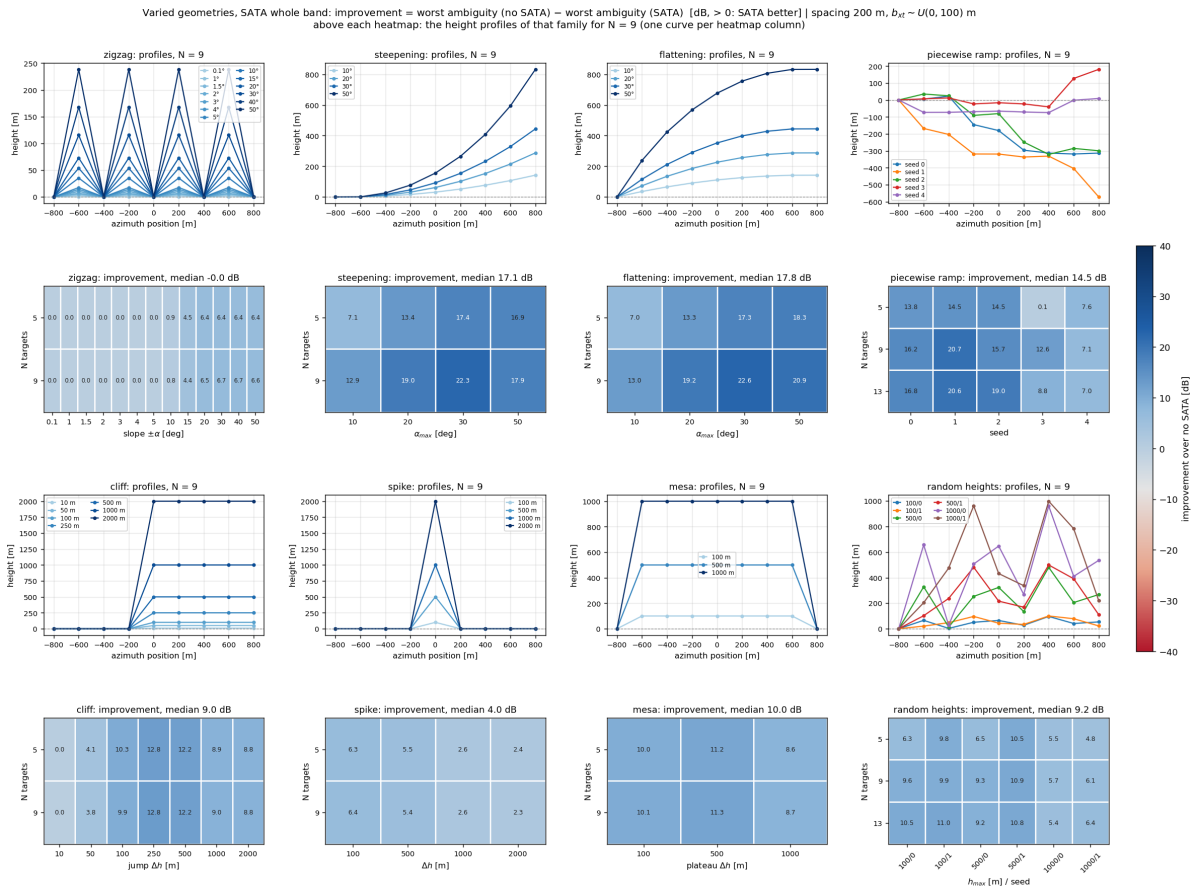


Figure 19: Other height profiles, SATA whole band. For each family, top: its height profiles for $N = 9$, one curve per heatmap column; bottom: improvement over no correction [dB], N targets against the parameter of the family.

Topographic ramp. The reference scenes of the report also use a topographic ramp: N targets at the *same* azimuth, on the line $\Delta h = \Delta y \tan \alpha$ through the scene centre, with Δy from 10 to 100 % of 2 km in ground range (`_make_topo_ramp` in `sar_recon/config.py`). It is now a family of the sweep (`topo_ramp`, $\alpha \in \{5, 10, 14.04, 15, 20, 30\}°$, $N \in \{5, 9, 13\}$). Because the targets are displaced in ground range and not in azimuth, their residuals include the flat-earth term and span a wide interval, from −1.6 to −16.0 cm on channel 0 for $N = 9$, $\alpha = 14.04°$. The 1D simulation sums all of them into one azimuth line, where they occupy a single azimuth cell. No map on that line can give them different values: the map holds one value there, the largest, and the correction is right for one target and wrong for the others. Fig. 20 shows the consequence. On the summed line SATA gains only 2.3 dB (median), sometimes less than nothing, exactly like the previous map, which reduces to the same single value. In a real acquisition, however, these targets are separated in slant range: by 18–520 m from the scene centre and by at least 13 m between neighbours, i.e. at least two range cells at 6 m resolution. They therefore lie in different range bins, and each range line is corrected with its own map (report, Section 4.3). Processing each target on its own line ("per range line") gives a median gain of 31.0 dB and reaches the oracle to within 0.03 dB in all 18 scenes (Fig. 21). The exception is $\alpha = 20°$, equal to the look angle: the ramp then follows the iso-range line, all targets fall within 3 m of slant range, in the same range cell as well as the same azimuth cell (layover), and no correction can separate them. Its per-range-line value is not attainable and is hatched in Fig. 20. The topographic ramp is therefore not a test of SATA 1D on a single line but of the range dimension, which is what the per-range-bin (2D) processing handles.

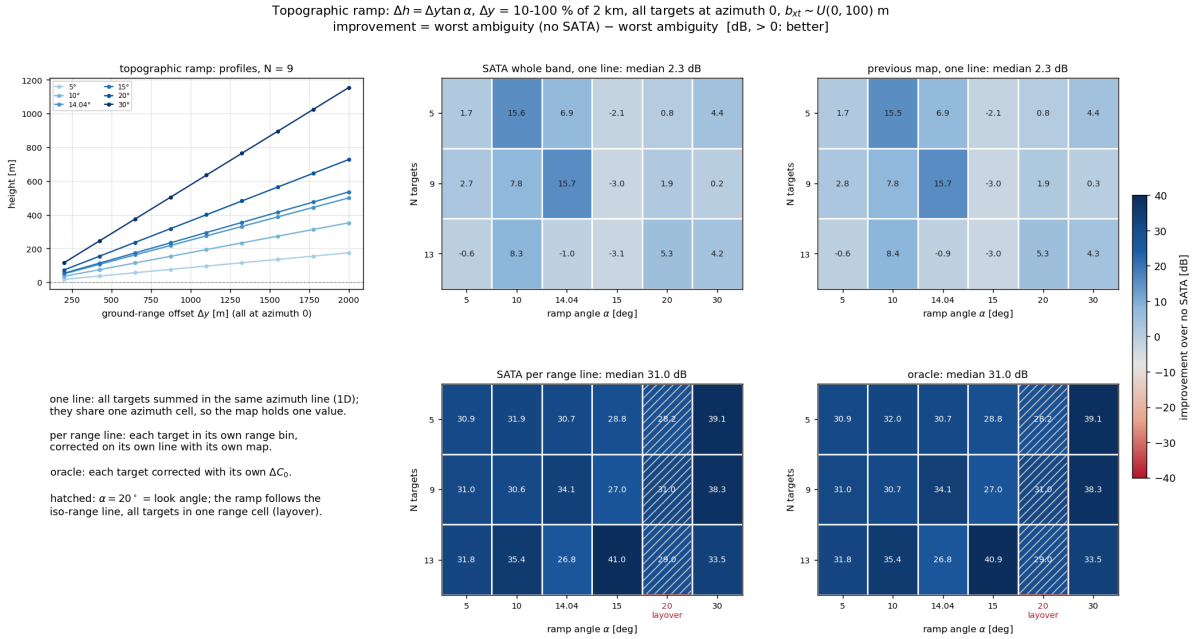


Figure 20: Topographic ramp. Left: height against ground-range offset for $N = 9$. Right: improvement over no correction [dB] with all targets summed in one line (SATA whole band, previous map), with each target corrected on its own range line, and for the oracle; hatched: $\alpha = 20°$, layover. The per-sub-band results are identical to the whole band to within 0.02 dB.

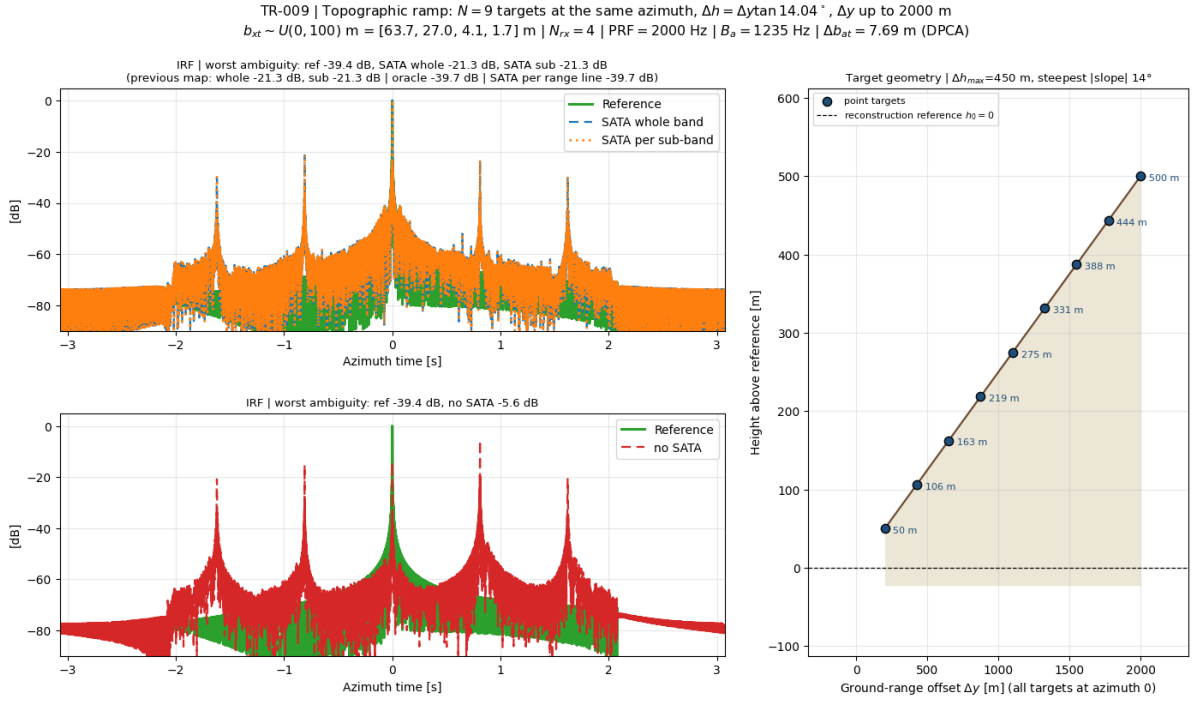


Figure 21: TR-009, the reference topographic ramp of the report ($N = 9$, $\alpha = 14.04°$). On one summed line SATA stops at -21.3 dB; per range line it reaches -39.7 dB, the oracle, against -39.4 dB for the reference.

Six scenes in detail. Figures 22–27 show one scene per regime, in the layout of the sweep script: top left the impulse response of the reference and of SATA (whole band and per sub-band), bottom left reference and no correction, right the target profile. The titles give the

worst ambiguities, including those of the previous map.

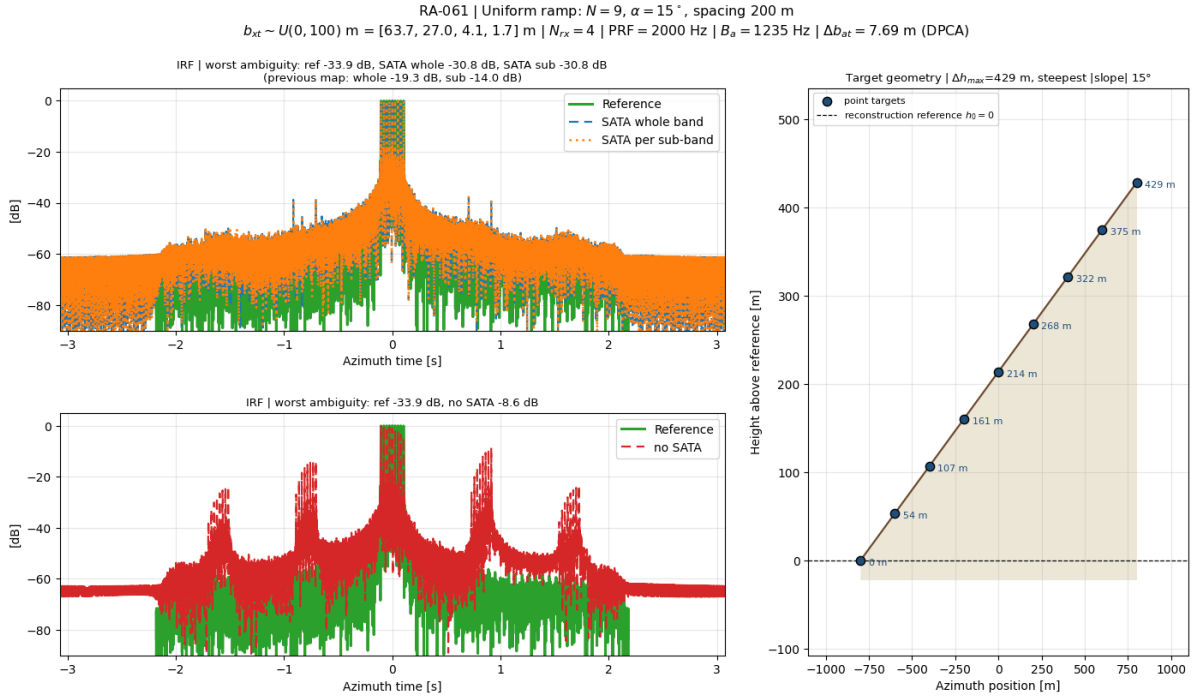


Figure 22: RA-061, ramp of nine targets at 15° (54 m between neighbours, 429 m in total). Smooth terrain: the interpolated block is the correct residual everywhere, and SATA brings the ambiguity from −8.6 to −30.8 dB, the oracle level; the previous map stops at −19.3 dB.

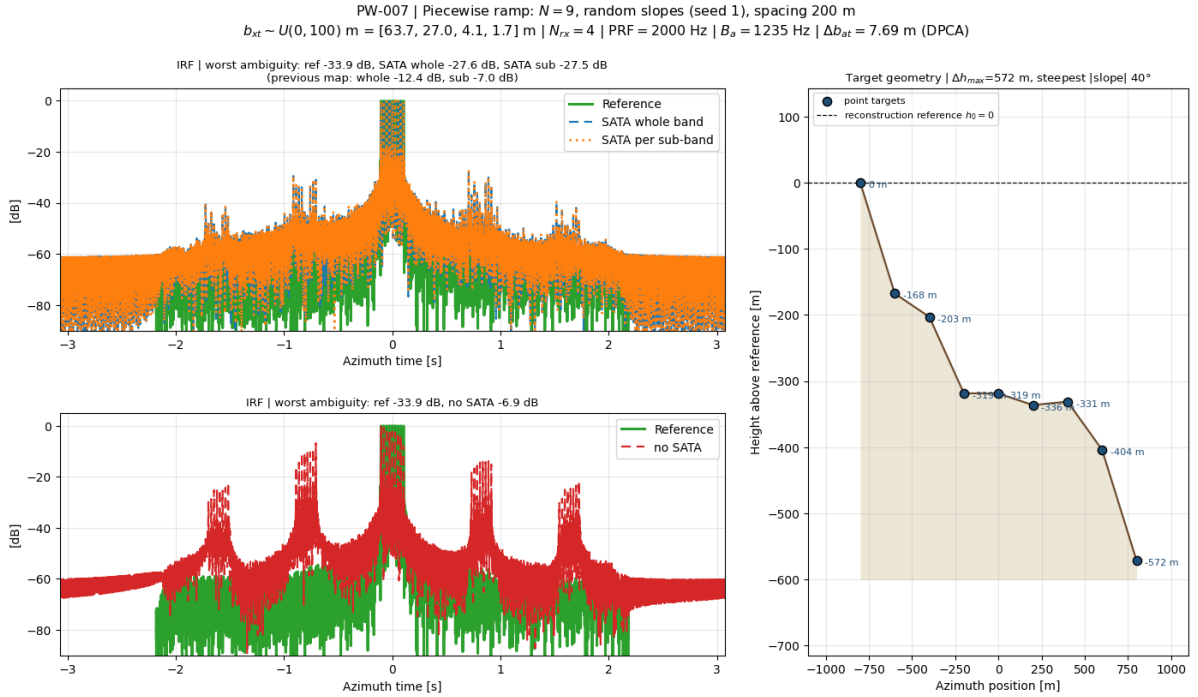


Figure 23: PW-007, piecewise ramp with random slopes up to 40° (jumps up to 168 m, 572 m in total). The kinks cost a few decibels with respect to the smooth ramp: −27.6 dB with SATA, against −12.4 dB with the previous map and −6.9 dB without correction.

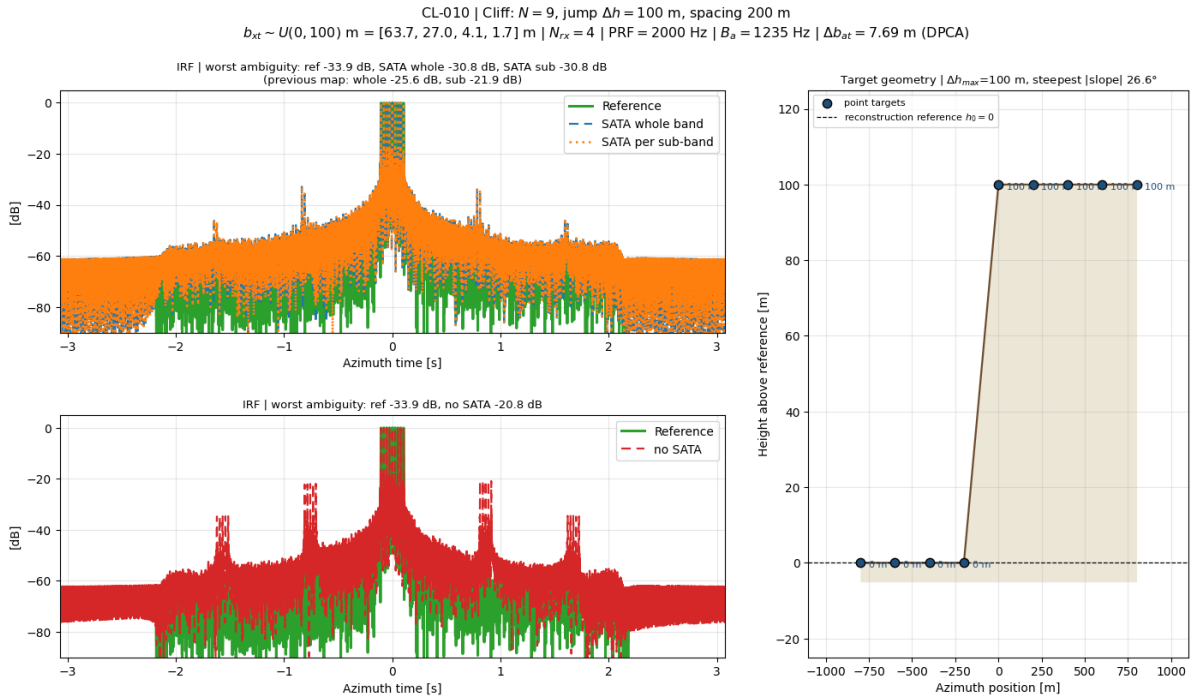


Figure 24: CL-010, cliff of 100 m in a line of nine targets. A single jump of 100 m is still at the limit of Fig. 17: SATA reaches −30.8 dB (oracle level), the previous map −25.6 dB.

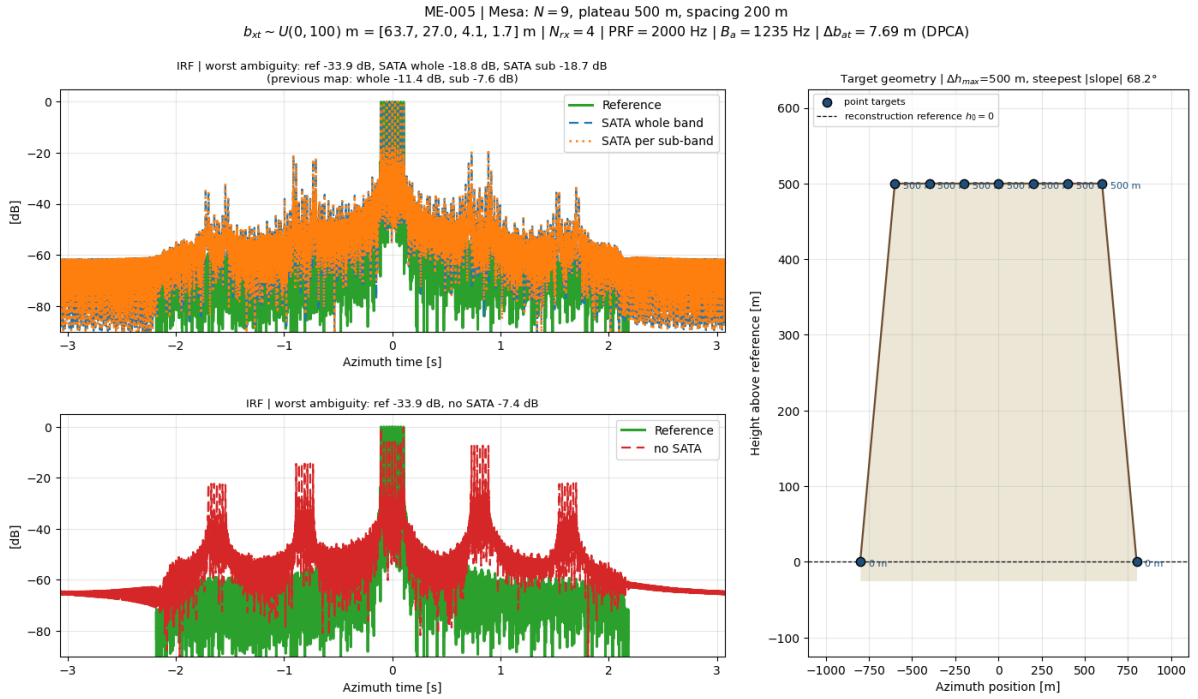


Figure 25: ME-005, mesa of 500 m (two jumps of 500 m). Beyond the limit: the ramps the interpolation draws across each jump are wrong for the targets on both sides. SATA still gains 11 dB over no correction (−18.8 against −7.4 dB) and 7 dB over the previous map, but stays 12 dB from the oracle level.

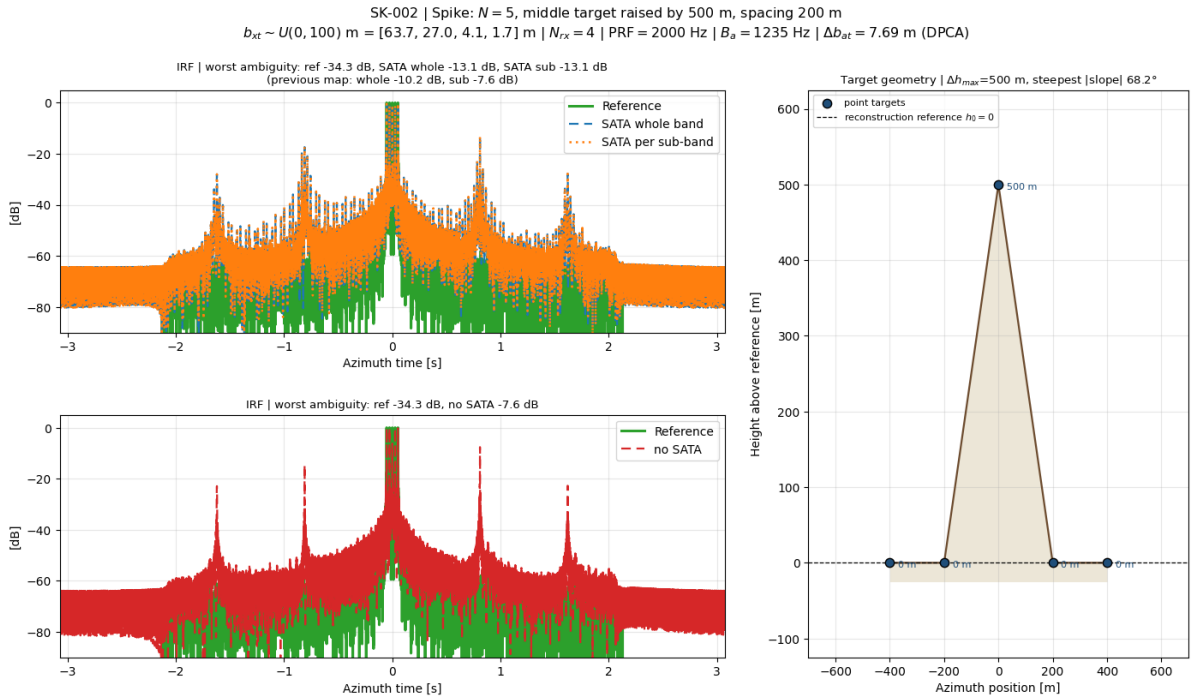


Figure 26: SK-002, one target 500 m above four at zero height, 200 m apart (the spike scene of T11). The central target's residual cannot be separated from its neighbours in any sub-aperture spectrum; SATA reaches −13.1 dB, the oracle −31.0 dB (Table 2).

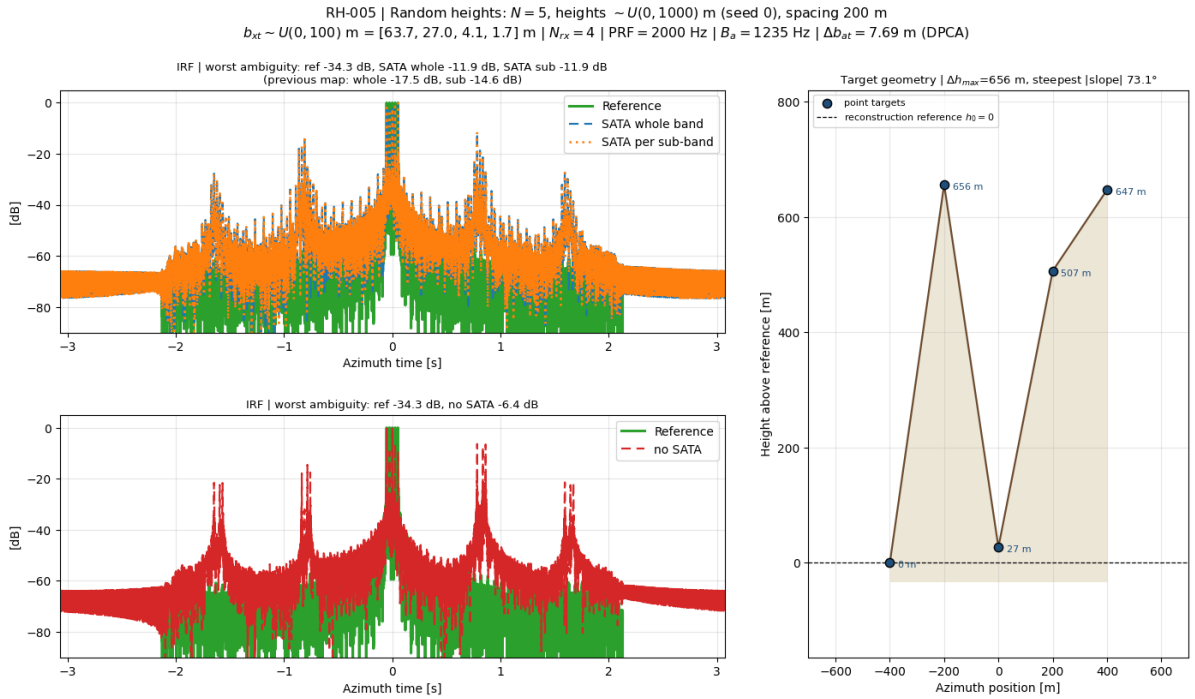


Figure 27: RH-005, five random heights up to 1 km (jumps up to 656 m), one of the three scenes where the previous map is better (−17.5 against −11.9 dB). Both are far from the oracle level: with jumps this large between targets 200 m apart neither map represents the scene, and which of the two wrong answers is less wrong depends on the particular profile. The other two such scenes (CL-006, CL-013) are cliffs of 1 km.

5.2 Consistency with the report

Table 4 lists the design points of Sections 3 and 4 of the report and where the implementation satisfies them.

Table 4: Checklist against the report.

Requirement	Implementation	Evidence
ΔC_0 from the same numerical model as the filters (Eq. 36)	`residual_C0` → `GetCoeffNu`	T4
$T_{\text{sub,eff}}$ from δx at the rate of the processed line (Eq. 44)	`subaperture_sizing` with PRF_{ch}	$T_{\text{sub,eff}} = 20$
triangular window, 50 % overlap, COLA self-test	`triangular_window`, `wsum`	T1
bin → position through $\beta(f)$ (Eq. 45), $\beta_{\text{img}} = 0$	`subaperture_axis`	T6
sub-band axis PRF_{ch} wide, centred on absolute f_k (Sec. 4.2, point 2)	`subband_axis` (corrected)	T7
broadside image grid $\beta_{\text{img}} = 0$ in the sub-bands (Sec. 4.2, point 3)	`subband_axis` (corrected)	T7
sub-band k reconstructed from the set corrected for k	`reconstruct_subband`	T10, T11
phase sign: remove $e^{-j2\pi\Delta C_0/\lambda}$	`inverse=True`	T3, T9
residual per channel at that channel's position ($b_{at}/2$)	`az_pixel_of_scatterer`	T5

6 Conclusions

The SATA 1D correction of ΔC_0 is implemented as the report describes, and the implementation is now verified piece by piece. The kernel reproduces its input for a null map and applies an exact global rotation for a constant one. The ruler places every STFT peak on its scatterer's cell, or on one of the predictable fold images $\pm X$. The map holds each scatterer's residual exactly where its energy lands, which realises the rule that a Doppler bin must be corrected with the residual of the scatterer it shows, and nothing else. For one target the ambiguities return to the reference level at every height. For several targets the correction matches the oracle whenever the scatterers are resolvable. On the 181 scenes of the geometry sweep, where the targets are 200 m apart and therefore never resolvable, the linear interpolation of the map holds the oracle level as long as neighbouring targets differ by less than about 100 m in height, and the new map beats the previous one by a median of 3.7 dB.

The per-sub-band kernel needed one correction: its frequency axis is now the absolute Doppler of the band on the broadside grid. The previous axis made it behave as the whole-band kernel. For the scenes simulated here both variants give the same result, because ΔC_0 of an iso-range target does not depend on the look angle. They will differ when the residual varies with angle within the aperture.

Three limits remain, and they belong to the method rather than to the code: scatterers closer than $\delta x \simeq 309$ m share one correction; an alias image of one scatterer that falls on the direct block of another is overwritten; and the linear and quadratic residuals ΔC_1, ΔC_2 are not corrected, which accounts for the 3 dB between oracle and reference on dense scenes. Beyond the simulation, the map requires the scatterer heights, which in practice come from a DEM.

A Running the Code

file	content
`sata1d/config.py`	system, scene, array; `make_config` (DPCA b_{at}, $b_{xt} \sim U(0, b_{xt}^{\max})$)
`sata1d/geometry.py`	transmitter and receiver tracks
`sata1d/signal_model.py`	point-target raw data, reference and channels
`sata1d/reconstruction.py`	`GetCoeffNu`, inversion filters, reconstruction
`sata1d/sata.py`	Stage A (map) and Stage B (`sata_1d`), whole band
`sata1d/subband.py`	per-sub-band fit, axis, kernel and reconstruction
`verify_sata1d.py`	tests T1–T11 of Section 5
`plot_irf_single_target.py`	Fig. 14
`plot_sata_windows_single_target.py`	window-by-window PDF of the kernel
`run_geometry_sweep.py`	geometry sweep (Section 5.1)

```
pip install -r requirements.txt
python verify_sata1d.py            # ~2 min, writes verification/
python plot_irf_single_target.py
python run_geometry_sweep.py --quick
python run_geometry_sweep.py --bxt 100 --workers 2   # full sweep, 181 scenes, ~5 min
```

Minimal use of the package on a scene of one's own:

```
import sata1d as S
cfg = S.make_config([(-1500.0, 240.0), (1500.0, -120.0)])   # (x [m], h [m]) per target
tr  = S.build_platform_tracks(cfg)
ch  = S.generate_channels(cfg, tr)
img_none  = S.reconstruct(cfg, tr, ch)
img_whole = S.reconstruct(cfg, tr, S.sata_channels(cfg, tr, ch))
img_sub   = S.reconstruct_subband(cfg, tr, ch)
```